

7. Parte VI: Do Circuito Integrado à Lua

A Década de 1960 viu o surgimento do circuito integrado, embora ainda computadores a válvula e transistor fossem construídos. Ela foi marcada pela invenção de várias linguagens de programação importantes como COBOL e BASIC.

Em 1961 o Brasil construiu seu primeiro computador, o Zezinho do ITA. Também nesta década a IBM lança a primeira família de computadores totalmente compatíveis do primeiro ao último modelo, iniciando a era da portabilidade e do upgrade indolor. Essa época viu a alvorada das redes de computadores, em especial, a invenção da comutação de pacotes e a construção da ARPANET, ancestral da Internet.

Conceitos de orientação a objetos começam a surgir, inicialmente com o Sketchpad e depois com Simula 67 e o projeto Dynabook. Nessa década a programação e engenharia de software tornam-se ciências mais sistemáticas, marcadas especialmente pelo tratado de Knuth sobre programação e as criações de Margareth Hamilton na área de engenharia de software, as quais permitiram que o homem fosse à Lua e voltasse com segurança.

7.1. Circuito Integrado – 1960

Ao longo da década de 1950 o uso do transistor sólido no lugar da válvula incandescente foi um grande avanço para a indústria dos computadores. Porém, à medida que as demandas por máquinas cada vez mais potentes cresciam, os construtores se deparavam com dificuldades imensas para montá-las, visto que um único ponto de solda mal feito poderia inviabilizar totalmente o funcionamento do computador, e eram centenas de milhares de soldas que precisavam ser feitas em cada máquina. Além disso, à medida que os computadores ficavam maiores havia o problema do tempo que os sinais elétricos levavam para atravessar os fios cada vez mais compridos. Essa limitação também inviabilizava a construção de computadores acima de um determinado tamanho.

Porém, a virada para 1960 viu nascer também um novo conceito: o circuito integrado, ou seja, a possibilidade de construir todo um conjunto de transistores a partir de um bloco único de material sólido, sem a necessidade de soldas e fios.

A primeira pessoa que teve essa ideia, pelo que se sabe, foi George W. A. Dummer (Reino Unido, 1909-2002). Ele trabalhava com engenharia de confiabilidade em um laboratório de pesquisas em telecomunicações. Seu trabalho era fazer com que circuitos baseados em transistores fossem cada vez mais confiáveis, o que era importante já que naquela época eles definitivamente não o eram. Ao verificar que seu trabalho acabava consistindo em reduzir cada vez mais o tamanho dos componentes, ele chegou à conclusão de que o próximo passo lógico seria produzir todo um conjunto de componentes a partir de um único bloco de cristal, como, por exemplo, silício. Em 1952 ele publicou essa ideia em uma conferência em Washington, sendo considerada a primeira publicação a mencionar circuitos integrados da história.

A ideia de Dummer era produzir várias camadas de elementos condutores, isolantes, retificadores e amplificadores, que seriam coladas juntas. O circuito em si seria construído a partir de recortes feitos nessas camadas, de forma adequada.

Apenas em 1957 ele conseguiu produzir uma primeira prova de conceito, ao construir um flip-flop a partir de um bloco sólido de semiconductor preparado para funcionar como quatro transistores. Porém, suas ideias não tiveram muita aplicação prática de início, porque os britânicos não perceberam seu potencial ou não quiseram assumir o risco do projeto.

Mas nos Estados Unidos, em 1958, havia pessoas dispostas a assumir esse risco. Um engenheiro chamado Jack Kilby (Estados Unidos, 1923-1995), estava trabalhando sozinho em seu laboratório na Texas Instruments, porque ele tinha sido recém contratado pela empresa e assim não tinha direito a férias na época em que a grande maioria de seus colegas saíram. A Texas tinha desenvolvido em 1954 o primeiro rádio transistorizado de bolso, o qual foi um tremendo sucesso e a empresa acreditava que as aplicações de circuitos desse tipo eram ilimitadas. Foi a Texas também que criou neste mesmo ano o transistor de silício, muito mais confiável do que o antigo transistor de germânio. Na mesma pesquisa eles descobriram que outros componentes como diodos, resistores e capacitores também podiam ser feitos de silício, o que foi fundamental para a concepção do circuito integrado, porque esse também precisaria desses outros componentes, não só transistores.

Como Kilby estava lá solitário, ele tinha muito tempo para pensar, e pensando ele chegou à conclusão de que poderia construir circuitos completos a partir de um único bloco. Essa ideia passou para a história como a “ideia monolítica”.

De início a ideia teve várias objeções. Já que o número de transistores normais que eram construídos e não passavam pelo controle de qualidade era absurdamente alto, construir uma peça única com vários transistores era, segundo as leis da probabilidade, uma certeza de falha.

Mas em 1960 ele conseguiu construir o primeiro circuito integrado, que tinha 11 por 1,5 milímetros e a Texas solicitou a patente do projeto neste mesmo ano. Uma réplica desse circuito é mostrada na **Figura Parte VI: Do Circuito Integrado à Lua-1**. A placa transparente inteira tem o tamanho aproximado de uma unha humana.

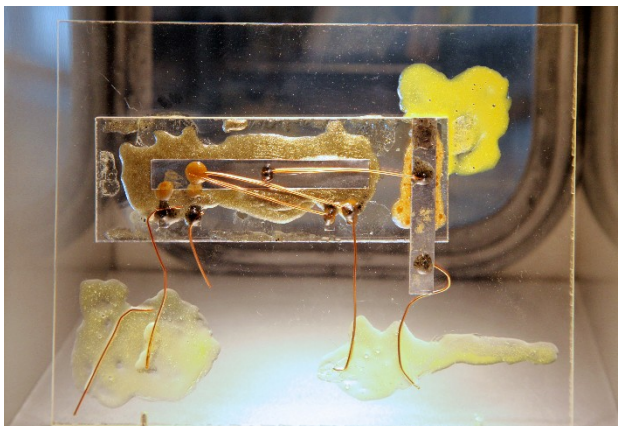


Figura Parte VI: Do Circuito Integrado à Lua-1: Réplica do primeiro circuito integrado a ser construído.¹ 1/2

Em 1959, porém, na Califórnia, outro inventor teve a mesma ideia de forma totalmente independente. Robert Noyce (Estados Unidos, 1927-1990) trabalhava na Fairchild Semiconductor, empresa que ele e mais 7 colegas fundaram em 1957. De fato, a ideia de Noyce era até superior à de Kilby, pois com ela era bem mais fácil construir os circuitos, que Noyce chamava de “circuito unitário” e Kilby de “circuito sólido”. A Fairchild entrou com um pedido de patente um pouco depois da Texas, o que levou Noyce a ficar sabendo dessa outra iniciativa. Ele procurou detalhar bastante a sua proposta, deixando claro que não infringia nenhuma das ideias de Kilby.

A patente de Noyce, embora tendo sido solicitada depois, foi concedida antes da de Kilby, em 1961; Kilby só receberia o reconhecimento em 1964. Assim, a Texas e a Fairchild entraram em uma disputa judicial, que só foi resolvida por um acordo em 1966. Ambas empresas reconheceram que de forma independente tinham inventado o circuito integrado. Elas poderiam produzir circuitos sem pagar direitos uma à outra, mas outras empresas que quisessem produzi-los teriam que pagar Royalties a ambas.

Hoje, tanto Kilby quanto Noyce são reconhecidos como os criadores do circuito integrado, mas o Prêmio Nobel de 2000 veio tarde para Noyce, que já havia falecido há 10 anos, e foi recebido por Kilby apenas.

A Texas começou a usar circuitos integrados em seus produtos ainda em 1960, e em 1963 o SN510 e SN514 foram os primeiros circuitos integrados a serem usados em órbita da Terra. A Fairchild também lançou seu primeiro circuito integrado comercial em 1960: um flip-flop. A partir de 1962 outras empresas começaram também a produzir esses circuitos, entre elas a Motorola e a Signetics.

7.2. COBOL – 1960

Em abril de 1959, um grupo de pessoas da academia, fabricantes e usuários de computador reuniu-se no centro de computação da Universidade da Pensilvânia. Entre outras coisas eles chegaram à conclusão de que era necessário rapidamente desenvolver uma linguagem de programação independente de hardware que fosse adequada para problemas da área empresarial. Para aplicações científicas já havia FORTRAN, e ALGOL estava também sendo definida. Mas a área comercial e empresarial precisava de sua linguagem. Assim nasceu a ideia para o desenvolvimento de COBOL.

Eles chegaram à conclusão de que necessitariam de um forte patrocínio para esse projeto e que este não poderia vir dos fabricantes de computadores, embora eles fossem convidados a participar da especificação da nova linguagem. Para obter um financiamento neutro, procuraram o Departamento de Defesa dos Estados Unidos, o DoD, que por sua vez ficou bastante interessado no projeto.

¹ "Replica IC" by Florian Schäffer - Own work. Licenced under CC BY-SA 4.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Replica_IC.png#/media/File:Replica_IC.png

Em maio de 1959 foi realizada assim, a primeira reunião especificamente convocada pelo DoD para a especificação do projeto da nova linguagem. Essa reunião aconteceu no Pentágono com aproximadamente 40 pessoas.

Inicialmente a nova linguagem era chamada de CBL (Common Business Language, ou Linguagem Comum de Negócios). A maioria dos participantes do evento concordou que deveria ser uma linguagem a mais próxima possível da língua inglesa, abrindo mão, dos formalismos matemáticos. O objetivo era que ela fosse compreensível por mais pessoas, além de matemáticos e engenheiros.

Porém, bem no início, o grupo sequer tinha certeza de que precisariam de fato criar uma nova linguagem. A ideia era, em curto prazo, analisar três outras linguagens que seriam candidatas a se tornar a CBL e talvez fossem propor uma combinação destas. As linguagens eram a FLOW-MATIC, desenvolvida por Grace Hopper da Sperry Rand, que, como vimos, era uma sucessora do A-0, a COMTRAN, ou Commercial Translator, definida pela IBM em 1959, mas nunca implementada e a AIMACO, ou Air Material Command, desenvolvida pela Força Aérea Americana também em 1959. Outras linguagens da época acabaram tendo uma menor influência, como Autocoder III, SURGE, FORTRAN, RCA 501 Assembler, Report Generator (GE Hanford) e APG-1 (Dupont). Os membros do comitê sabiam dos desenvolvimentos de ALGOL, mas consta que apesar disso ela não teve nenhuma influência no seu trabalho.

As empresas RCA e Sylvania, que participavam do grupo, também estavam desenvolvendo suas linguagens por essa época. Assim, um dos desafios iniciais do grupo era estancar esses projetos, de forma que todos se concentrassem na linguagem comum.

Foi então estabelecido um comitê executivo, o CODASYL (Committee on Data Systems Languages), o qual supervisionaria o trabalho de outros três comitês, o de curto prazo, o de médio prazo e o de longo prazo. O comitê de longo prazo acabou nunca sendo efetivamente criado. E o de curto prazo fez um trabalho em tempo recorde esperando que este seria apenas um trabalho provisório a ser totalmente revisado pelo comitê de médio prazo. Mas isso não aconteceu e assim, o resultado do trabalho do comitê de curto prazo acabou se consolidando.

O grupo incluía representantes de várias empresas que eram competidoras entre si. Comenta-se que as questões políticas sempre afloravam nas reuniões. Cada empresa concordava em implementar a linguagem, mas sempre havia um “mas”, com o qual era difícil obter consenso. Além disso, foram criados dois comitês técnicos, um para projetar a linguagem de definição de dados e outro para projetar a linguagem de comandos. Essas duas divisões na linguagem, como se fossem programas separados são uma das características específicas de COBOL que não é compartilhada por quase nenhuma outra linguagem de programação, pois na maioria das linguagens, dados e comandos são definidos conjuntamente, mas em COBOL são documentos completamente separados.

Em agosto de 1959 os comitês decidiram que os programas escritos nessa linguagem teriam três grandes divisões: dados, procedimentos e ambiente. A primeira definindo as

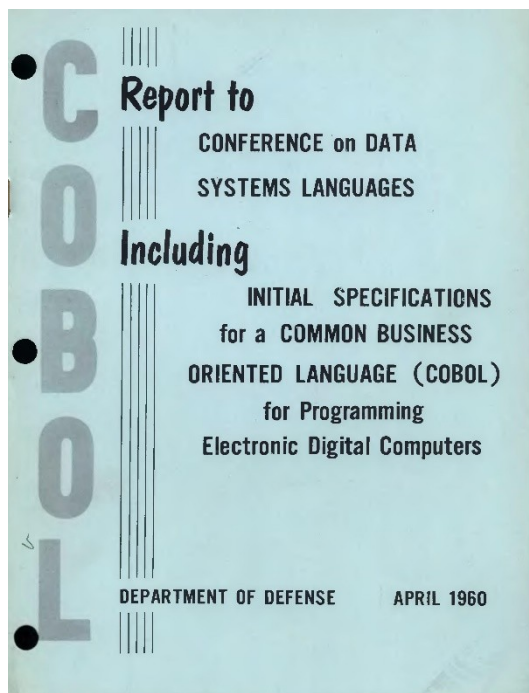
estruturas dos dados, a segunda definindo as operações a serem feitas e a terceira a especificação do hardware no qual o programa seria compilado e executado.

Numa reunião em setembro vários nomes foram propostos para a nova linguagem. Entre eles:

- BUSY: Business System.
- BUSYL: Business System Language.
- INFOSYL: Information System Language.
- DATASYL: Data System Language.
- COSY: Common Systems Language.
- COCOSYL: Common Computer Systems Language.

Felizmente, o nome escolhido acabou sendo COBOL, como sigla para Common Business Oriented Language.

Finalmente, em dezembro de 1959, após muitos dias e noites e trabalho intenso dos comitês, o Relatório COBOL ficou pronto, e foi publicado no início do ano seguinte (Figura Parte VI: Do Circuito Integrado à Lua-2), sendo o marco para o início da implementação da linguagem. Em 1960 os principais fabricantes de computadores, então, entraram na corrida para ver quem desenvolvia primeiro o compilador COBOL. Por indefinições internas, a IBM ficou de fora dessa corrida por algum tempo. Enquanto isso, a RCA e a Remington-Rand-Univac terminaram seus compiladores. Em dezembro de 1960 um marco histórico: dois computadores de fabricantes diferentes rodaram o mesmo programa em suas máquinas, a única diferença entre os programas teria sido a divisão de ambiente, a qual especificava a máquina onde o programa iria funcionar e por isso mesmo tinha que ser diferente. A era da portabilidade de sistemas iniciava assim.



Pode-se dizer que o trabalho de Hopper com FLOW-MATIC teve grande influência em COBOL. Jean E. Sammet (Estados Unidos, 1928), outra mulher que participou do comitê de criação de COBOL, relembra as principais influências que FLOW-MATIC teve em COBOL:

- FLOW-MATIC funcionava! Ela já estava em uso em várias empresas e organizações militares.
- Ela permitia usar nomes longos. Mesmo ainda estando limitada a 12 caracteres, que era o tamanho de uma palavra no UNIVAC, era melhor escrever SOCIAL-SECUR do que SOSEC, como seria em FORTRAN.
- Ela usava palavras em inglês em vez de símbolos matemáticos. Ler expressões com palavras como ADD ou COMPARE era bem mais fácil do que ler símbolos matemáticos para quem não estava acostumado com eles.
- FORTRAN considerava todas as palavras de dados em memória como tendo o mesmo tamanho. Isso podia funcionar com aplicações científicas, mas não era adequado para aplicações comerciais. As vezes se precisava de um único caractere para indicar M ou F, representando o gênero masculino ou feminino. Em outros casos era necessário definir dados longos, maiores do que a palavra que o computador usava. Assim, tanto FLOW-MATIC quanto COBOL permitiam a definição de dados de tamanhos variados, independentemente da máquina que fosse processar estes dados.

Em alguns casos, ela confessa, as decisões sobre nomes de comandos foram tomadas especificamente para serem diferentes do FORTRAN da IBM. Por exemplo, em vez de usar a palavra “DO” para indicar um comando de repetição, eles preferiram usar “PERFORM”, simplesmente porque FORTRAN usava “DO”.

COBOL permitia que qualquer instrução fosse finalizada com a sequência “AND GO TO”, seguido de um endereço, o que permitia que o fluxo de controle do programa fosse redirecionado para qualquer outro ponto rotulado pelo endereço. Com o passar do tempo, GOTO tornou-se um comando da maioria das linguagens de programação. Mas com a adoção das técnicas de programação estruturada, a partir da década de 1970, cada vez mais o uso do GOTO tornou-se heresia em programação e símbolo do chamado “código macarrônico”, pois são tantos desvios para lá e para cá no código que a organização dele mais se parece com um prato de macarrão.

COBOL exigia que cada comando iniciasse com um verbo. Isso obrigava, por exemplo, a escrever “COMPUTE” antes de uma fórmula que deveria ser calculada. Mas havia exceções, porque, por exemplo “IF” não é um verbo, mas pode iniciar um comando.

COBOL também permitia que o programador usasse o verbo DEFINE para definir o significado de um novo verbo como sendo uma sequência de comandos. Assim, essa foi possivelmente a primeira vez na história que uma linguagem de programação implementava o conceito de “macro”, ou seja, a possibilidade de dar nome a um

² Imagem de domínio público disponível em: https://en.wikipedia.org/wiki/File:COBOL_Report_Apr60.djvu

fragmento de código, sendo que esse nome, cada vez que fosse usado no código seria substituído pelos comandos usados para defini-lo.

A linguagem não tinha funções, porque, para o comitê, isso só teria interesse em aplicações científicas, já que na época o conceito de função era associado apenas com seno, cosseno, etc.

Outra característica única em COBOL, que visava a legibilidade era a possibilidade de escrever palavras que não tinham nenhum significado para o compilador, mas que tornavam os comandos mais legíveis. Por exemplo, o programador poderia escrever, usando apenas as palavras essenciais:

```
READ file-name INTO data-name; END any-imperative-statement.
```

Ou poderia usar uma forma mais legível usando *noise words*, isso é, palavras que não eram traduzidas em comandos:

```
READ file-name RECORD INTO data-name; AT END any-imperative-statement.
```

Outra característica interessante que não se vê na maioria das linguagens é a possibilidade de escrever expressões contextualizadas. Se você vai fazer dois testes ou mais, na maioria das linguagens vai escrever algo como:

```
IF X IS EQUALS TO Y comando1;  
IF X IS GREATER THAN Y comando2;  
IF X IS LESS THAN Y comando3.
```

Em COBOL você poderia escrever:

```
IF X IS EQUALS TO Y comando1;  
IF GREATER comando2;  
OTHERWISE comando3.
```

Hoje, COBOL é considerada uma linguagem antiquada pela maioria dos programadores, sendo difícil achar jovens programadores que se habilitem a estudá-la. Apesar disso, foi estimado na virada do Século XX para XXI que haveria no mundo cerca de 200 bilhões de linhas de programas escritas em COBOL e que elas seriam responsáveis por cerca de 80% dos sistemas comerciais no mundo. Este fato, aliado ao fato de que muitos programadores nos anos 1960 e 1970 optavam por representar o ano, nas datas, com apenas 2 caracteres é que levou ao famoso “bug do milênio”, como veremos. Como o ano 1999 estava representado na maioria dos sistemas simplesmente como “99”, quando fosse meia noite de 31 de dezembro desse ano, muitos sistemas passariam a acreditar que estavam no ano “00”, mas isso não significava 2000 para os computadores e sim 1900. Assim, no final do Século XX uma intensa busca por programadores COBOL aconteceu no mundo para corrigir os antigos sistemas ainda em uso e prepará-los para usar 4 dígitos em vez de 2.

7.3. DEC PDP-1, o Primeiro Minicomputador – 1960

Vimos que o Laboratório Lincoln do MIT desenvolveu o TX-0, um dos primeiros computadores totalmente transistorizados. Com a evolução do projeto para o TX-1 e depois o TX-2 acabaram acontecendo algumas dificuldades que fizeram com que vários membros da equipe saíssem do MIT para fundar sua própria empresa, entre eles Ken Olsen (Estados Unidos, 1926-2011) e Harlan Anderson (Estados Unidos, 1929).

Foi assim fundada a Digital Equipment Corporation (DEC) que passou a produzir um computador cujo design se baseou em uma versão mais “limpa” do TX-2. Esse novo computador começou a ser entregue aos clientes em 1960. A máquina chamava-se Programmed Data Processor-1 ou PDP-1 (Figura Parte VI: Do Circuito Integrado à Lua-3). O PDP-1 é também considerado o primeiro minicomputador a entrar em operação comercialmente.

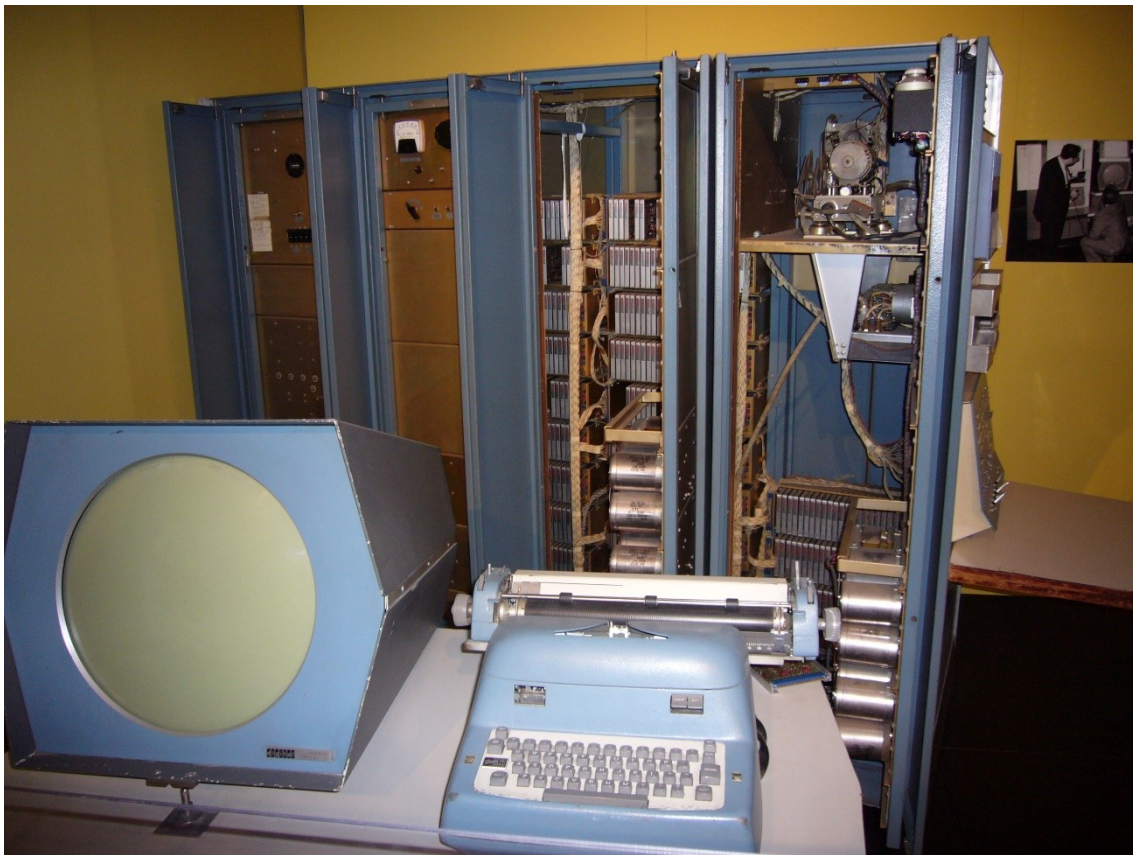


Figura Parte VI: Do Circuito Integrado à Lua-3: PDP-1.³ 1/1

A DEC procurou valorizar em seus computadores um fenômeno observado por seus criadores no MIT. Eles perceberam que apesar de haver uma máquina IBM muito potente disponível, os estudantes faziam fila para usar o TX-0, mesmo este já estando praticamente depenado por esta altura. Ocorre que o TX-0 era um computador que permitia interatividade com o usuário enquanto que as máquinas IBM eram meros processadores de dados.

³ "PDP-1" by Matthew Hutchinson - <http://www.flickr.com/photos/hiddenloop/307119987/>. Licensed under CC BY 2.0 via Wikimedia Commons - <https://commons.wikimedia.org/wiki/File:PDP-1.jpg#/media/File:PDP-1.jpg>

Em 1962 a DEC doou uma unidade PDP-1 para o MIT, a qual foi colocada na mesma sala de seu ancestral TX-0. O PDP-1 logo se tornou a máquina preferida da crescente comunidade *hacker* do MIT. Esse computador foi o palco de alguns momentos importantes da história da computação como o jogo Spacewar!, o primeiro editor de texto, o primeiro *debugger* interativo, um jogo de xadrez completo e vários sistemas relacionados à música computacional.

O PDP-1 era vendido a 120 mil dólares (equivalente a aproximadamente 950 mil dólares em 2015). Cinquenta e três máquinas foram produzidas até 1969. Três delas ainda existem e estão expostas no museu de história da computação de Mountainview, Califórnia. Uma das máquinas ainda é totalmente operacional e é ligada duas vezes por mês para que os visitantes do museu possam vê-la funcionar.

O PDP-1 tinha um monitor CRT com resolução de 1024x1024 pontos. Ele tinha comandos para desenhar pontos, retas e mesmo curvas neste monitor, o que fez com que se tornasse uma máquina fundamental para o desenvolvimento inicial da computação gráfica.

Havia também uma espécie de caneta que podia ser colocada na tela e que era capaz de identificar qual ponto da matriz de 1024x1024 estava tocando. Assim, esse instrumento podia ser usado para desenhar na tela e obter representações digitais desses desenhos.

Ele foi também bastante usado para a produção de música computacional. Com alguns periféricos especialmente construídos, ele era capaz de tocar músicas com harmonia de 4 vozes, contando inclusive com um sofisticado programa de codificação de música, o Harmony Compiler que foi usado para executar muitas composições de Bach e de Mozart.

7.4. IFIP - International Federation for Information Processing – 1960

Em junho de 1959 a UNESCO, Organização das Nações Unidas para a Educação, a Ciência e a Cultura, organizou em Paris a Primeira Conferência Internacional em Processamento de Informação. Essa conferência, hoje rebatizada como IFIP World Computer Congress, que ocorre a cada 4 ou 5 anos, é considerada como o primeiro congresso da IFIP – International Federal for Information Processing.

Assim, no ano seguinte, 1960, A UNESCO constitui a federação internacional de sociedades científicas da área de computação. Originalmente ela foi batizada como IFIPS – International Federal for Information Processing Societies, mas em 1961 o termo “Societies” foi eliminado e a organização passou a se chamar simplesmente IFIP.

A IFIP funciona como uma federação de sociedades científicas. O Brasil já foi representado na IFIP pela SUCESU (Associação dos Usuários de Informática e Telecomunicações), mas atualmente é representado pela SBC (Sociedade Brasileira de Computação). A ACM é membro da IFIP também desde seu início.

A sociedade organiza seus trabalhos em torno de 13 comitês técnicos, cada um deles dedicado a um tópico diferente, como por exemplo, educação (TC3), sistemas de

informação (TC8), inteligência artificial (TC12), etc. cada um dos comitês técnicos tem um representante de cada uma das sociedades científicas e um representante de seus grupos de trabalho (WG), usualmente o coordenador ou subcoordenador do grupo.

Já os WG se organizam em pequenos grupos de profissionais interessados em diferentes subtópicos do TC. Por exemplo, o TC12 (inteligência artificial) tem nove grupos de trabalho, como por exemplo, agentes inteligentes (WG12.3), bioinformática inteligente e sistemas biomédicos (WG12.8) etc.

As regras para pertencer a um grupo de trabalho variam, mas usualmente exige-se que o interessado já tenha participado de pelo menos dois eventos científicos (simpósios, congressos ou workshops) organizados pelo grupo de trabalho, quando então ele pode participar de uma reunião do grupo de trabalho e pode ser indicado como membro correspondente (sem direito a voto). Quando o membro correspondente participar de sua segunda reunião do grupo de trabalho ele pode ser indicado como membro permanente.

A IFIP tem atuado ativamente no desenvolvimento da computação desde seu início. Como maiores destaques pode-se mencionar o estabelecimento da linguagem ALGOL que, embora não seja mais muito utilizada atualmente, produziu as bases teóricas para a grande maioria das linguagens imperativas dos dias de hoje, e o framework de currículos de referência na área de informática, um documento que procura orientar universidades, especialmente aquelas em regiões com maior dificuldade de recursos, na elaboração de currículos de informática objetivos e factíveis. Além disso, a federação organiza regularmente eventos e publicações que são acessíveis gratuitamente através de sua biblioteca digital.

7.5. Compartilhamento de Tempo – 1961

Até 1960 os computadores eram usados por um usuário de cada vez. Mas como os computadores ainda eram muito caros, qualquer minuto ou mesmo segundo que ele ficasse inativo seria um desperdício de recursos. A invenção dos sistemas operacionais diminuiu significativamente o tempo de ociosidade entre um usuário e outro. Mas enquanto um usuário estivesse usando a máquina poderia haver momentos de ociosidade até que a máquina fosse liberada para o usuário seguinte.

A forma de aproveitar ao máximo a máquina era permitir que vários usuários a usassem ao mesmo tempo. Dessa forma, com vários programas rodando ao mesmo tempo, quando um deles ficasse ocioso ou deixasse de precisar do processador momentaneamente, o tempo poderia ser ocupado por outro programa. Na prática, os sistemas operacionais tiveram que ser adaptados para permitir que vários programas fossem acionados ao mesmo tempo. Assim, o tempo de processamento passou a ser dividido em “fatias”. Cada programa poderia usar o processador apenas durante a sua fatia. Depois o computador passaria a executar a fatia do programa seguinte, e assim por diante, até que o primeiro tivesse novamente direito à sua fatia.

A primeira publicação deste conceito ocorreu em 1957 em um artigo de Bob Bemer (Estados Unidos, 1920-2004). Mas o primeiro projeto a colocar essa ideia em prática foi iniciado por John McCarthy (o criador do LISP) no final de 1957. Assim, embora

McCarthy tenha saído do projeto depois, o CTSS ou Compatible Time-Sharing System, o primeiro sistema operacional com compartilhamento de tempo foi demonstrado em 1961 no MIT.

Ele teve a ideia quando foi trabalhar no MIT em 1957 e teve contato com um IBM 704. Porém, para colocá-la em prática, teria que criar um sistema de interrupção para essa máquina. Uma “interrupção” era a possibilidade de fazer o computador parar o que estava processando para fazer outra coisa e depois eventualmente voltar para o programa anterior.

Porém, antes que ele instalasse o sistema que havia concebido ele soube que a IBM tinha desenvolvido um “pacote para tempo real”, a pedido da Boeing, que usava um 704 para captar informações produzidas por um túnel de vento. O pacote foi requisitado e, embora demorasse mais de um ano, foi finalmente entregue. A ideia era operar o 704 a partir de 3 terminais simultaneamente.

Além disso, a IBM concordou em fazer modificações no novo computador 7090 que seria instalado no MIT. Essas modificações incluíam mais memória para o sistema de compartilhamento de tempo e proteção de memória que era uma forma de evitar que um programa interferisse com outro quando estivessem rodando ao mesmo tempo na máquina.

O PDP-1, sobre o qual já falamos, foi o primeiro computador a disponibilizar compartilhamento de tempo como uma característica de fábrica.

McCarthy defendeu muito a ideia de sistemas de compartilhamento de tempo. Ele propunha que tempo de computador deveria ser tratado como serviço público, ou seja, como água e luz, que você tem a disposição e paga conforme o uso. A ideia dele foi marcante para a indústria até a década de 1980 quando o surgimento dos microcomputadores diminuiu a necessidade de sistemas de compartilhamento de tempo, porque então cada um poderia ter seu próprio computador a um preço relativamente baixo. Na década de 2010, porém, a ideia volta novamente com muita força, com a chamada “computação em nuvem”.

7.6. Videogame Spacewar! – 1961

“Spacewar!” não foi o primeiro videogame, como já vimos, mas certamente foi o primeiro a ser amplamente utilizado. Ele foi desenvolvido para o computador interativo, o PDP-1 (Figura Parte VI: Do Circuito Integrado à Lua-4). O jogo era para dois jogadores, cada um com uma nave que disparava mísseis contra a outra. No centro da tela havia uma estrela, cuja gravidade atraía as naves, mas não os mísseis, por falta de poder de processamento.

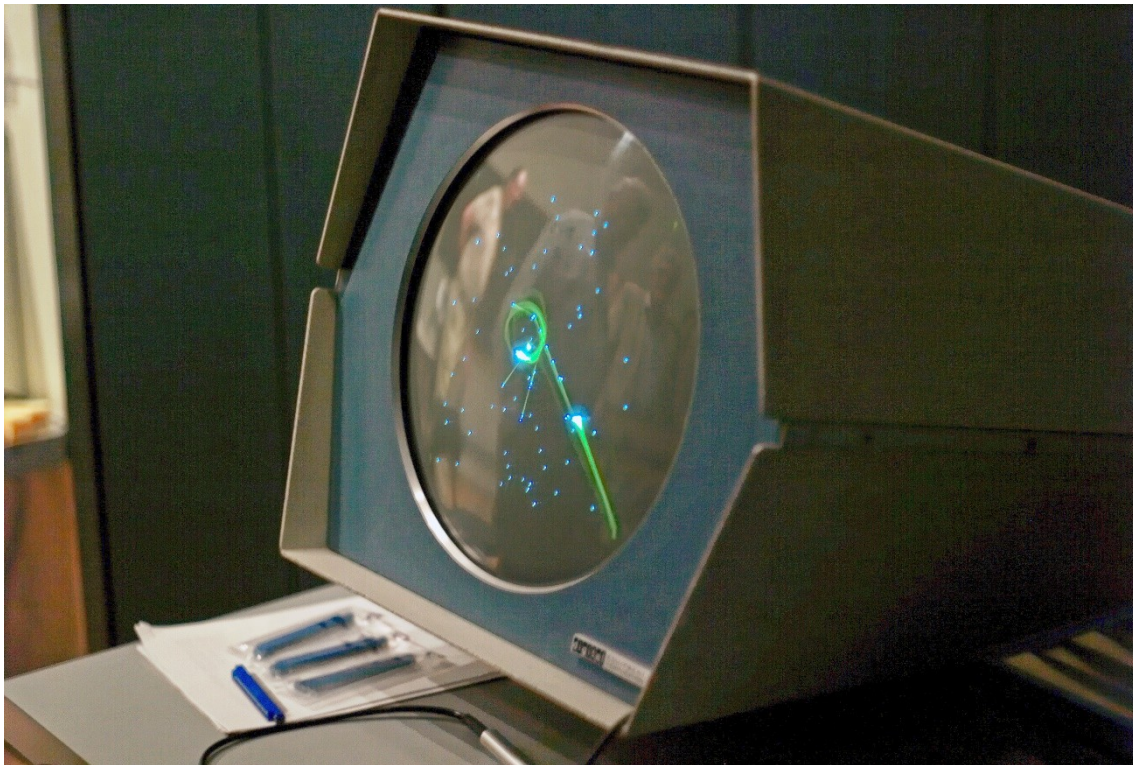


Figura Parte VI: Do Circuito Integrado à Lua-4: Spacewar! no PDP-1.⁴ 1/1

Em uma situação de emergência, um dos jogadores poderia ir para o hiperespaço. Isso fazia sua nave desaparecer e reaparecer em um lugar aleatório na tela. Mas cada vez que o recurso do hiperespaço era usado, a probabilidade de a nave explodir aumentava.

Um dos principais autores do Spacewar! foi Steve Russel (Estados Unidos, 1937), o mesmo que implementou o LISP para o IBM 704. O jogo foi implementado em 1961 no PDP-1 do MIT.

Russel conta que os estudantes do MIT estavam motivados a fazer programas interessantes que pudessem demonstrar as capacidades do novo computador PDP-1, que substituiria o antigo TX-2. Com a nova tela de CRT, pensaram em fazer algo que fosse visualmente manobrável. A escolha óbvia, segundo ele, seriam naves espaciais.

Russel foi apelidado de “lesma” pelos seus colegas, devido à sua demora em executar as tarefas de programação. Em relação ao Spacewar!, conta-se que ele procrastinava muito, alegando, entre outras coisas que não tinha as rotinas para calcular senos e cossenos que seriam necessárias para implementar o jogo. Seu colega, Alan Kotok (Estados Unidos, 1941-2006) entrou em contato com a DEC, que disse que tinha as rotinas. Kotok dirigiu seu carro até lá, pegou uma fita com as rotinas, jogou-a na frente de Russel e perguntou qual a desculpa agora.

⁴ "Spacewar!-PDP-1-20070512" by Joi Ito from Inbamura, Japan - Spacewar running on PDP-1. Licensed under CC BY 2.0 via Wikimedia Commons - <https://commons.wikimedia.org/wiki/File:Spacewar!-PDP-1-20070512.jpg#/media/File:Spacewar!-PDP-1-20070512.jpg>

Russel então começou a codificar o jogo, iniciando em dezembro de 1961 e finalizando em 1962. Spacewar! ainda pode ser jogado no PDP-1 em operação no Museu de História da Computação na Califórnia.

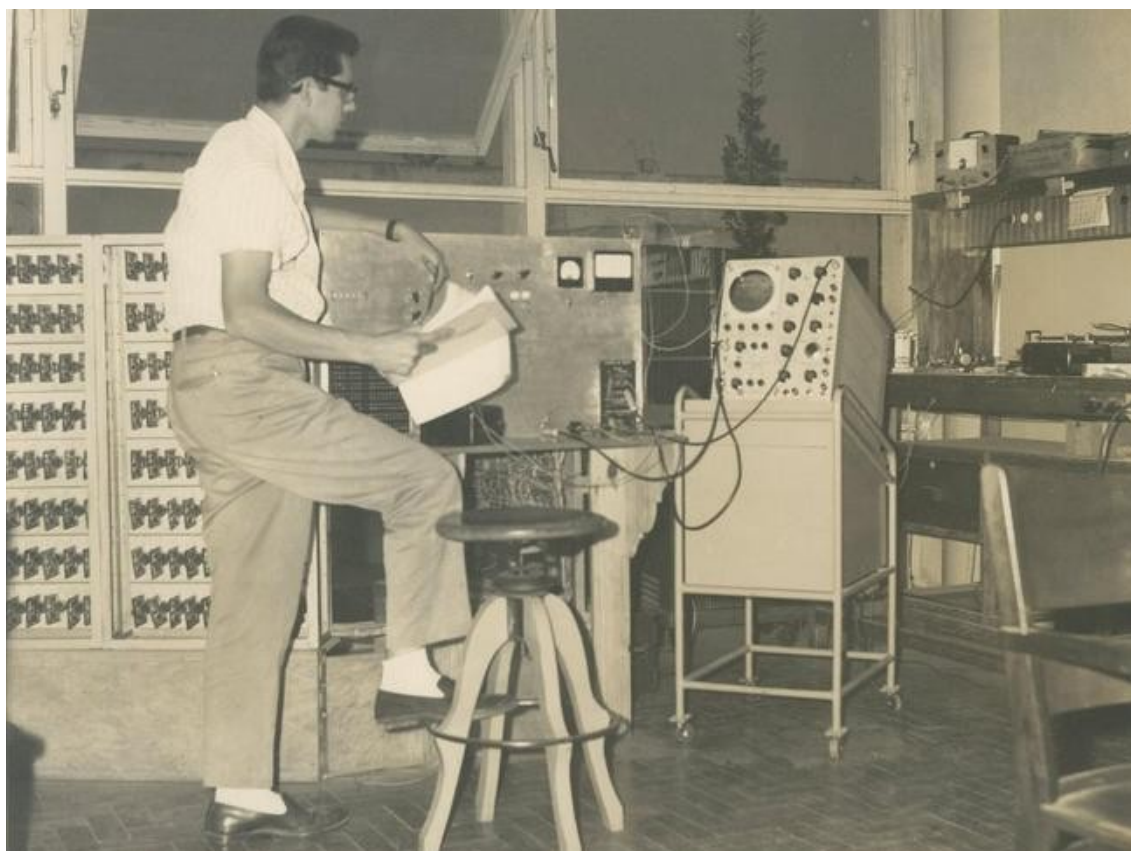
7.7. Zezinho, o Primeiro Computador Brasileiro – 1961

Era 1961 e o Brasil construía seu primeiro computador, o Zezinho. Ele foi construído no Instituto Tecnológico da Aeronáutica (ITA).

Tercio Pacitti (Brasil, 1928-2014) introduziu o primeiro computador no Brasil, um IBM 1620. Pacitti ficou muito conhecido também por ter sido autor do livro “FORTRAN Monitor”, que vendeu cerca de 250 mil cópias. Houve muita burocracia para trazer o computador para o Brasil. No final ele entrou sob a rubrica de “material para segurança nacional”. O computador, doado pela Fundação Ford, foi usado unicamente para fins didáticos.

Esse fato elevou o moral dos professores americanos do instituto. Porém, os professores europeus instigavam o diretor para que criasse o projeto de um computador brasileiro, “mais potente do que o norte-americano”.

Iniciou então o projeto do Zezinho (**Figura Parte VI: Do Circuito Integrado à Lua-5**), que foi realizado por quatro alunos de graduação da turma de 1961: Alfred Volkmer, András György Vásárhelyi, Fernando Vieira de Souza e José Ellis Ripper Filho. Eles foram orientados pelo professor Tien Wei Chu.



O computador foi construído também com finalidade didática, não havendo interesse na época em iniciar uma indústria nacional de computação, o que só veio a acontecer mais de uma década depois.

O Zezinho tinha uma memória de 100 bits apenas, e uma velocidade muito baixa. Em 1963 ele foi aperfeiçoado pelo formando Valdemar W. Setzer (Brasil, 1940), que projetou uma nova fonte de alimentação, e adquirindo então sua forma final.

7.8. ReserVec, Sistema de Reserva de Passagens Aéreas – 1962

A década de 1950 viu o boom da aviação comercial. Porém, naqueles tempos, conseguir fazer uma reserva de passagem aérea era um verdadeiro sacrifício que chegava a deixar uma pessoa pendurada no telefone por até 90 minutos. Cada empresa tinha uma central de reservas que registrava os voos e seus passageiros em fichas de papel que ficavam adequadamente estocadas em gavetas. O problema é que ter muitos funcionários procurando fichas em gavetas acabava causando dificuldades de acesso aos armários, e demasiados tempos de espera. E quando o voo estava cheio, o funcionário tinha que voltar ao telefone, pedir outra opção de data ou voo e retornar às gavetas.

Já em 1946 a American Airlines criou um dispositivo eletromecânico para minimizar esse tempo, o “Reservisor”, que era uma máquina que permitia a um funcionário selecionar em uma matriz o número do voo e o dia (inicialmente a máquina permitia fazer reservas para os próximos 10 dias) e marcar que um passageiro estava solicitando reserva neste voo. A partir desse registro os funcionários conseguiam saber o status de um voo sem sair de sua mesa.

Com isso a empresa conseguiu aumentar bastante o número de atendimentos. Infelizmente, o sistema não funcionava muito bem. O maior problema eram os relês, que ficavam sujos, evitando o contato correto, e precisavam ser limpos de tempos em tempos. Muitos contratemplos aconteceram neste período.

Porém, não foi a American a primeira empresa a lançar um verdadeiro sistema de reserva de passagens aéreas. Essa honra coube à TCA (Trans-Canada Airlines), hoje Air Canada, que começou a usar em 1962 o “ReserVec”, ou Reservations Electronically Controlled. A TCA conhecia o Reservisor, mas como ele falhava muito, eles não acharam o projeto atraente. Um de seus engenheiros passou a estudar o problema em 1952 e chegou à conclusão que a única solução seria usar computadores.

A TCA procurou ajuda da Universidade de Toronto, que possuía um computador Manchester Mark I chamado FERUT. Um sistema de reservas foi implementado no FERUT e avaliado em 1953. O sistema funcionava bem, mas a entrada e saída de dados era ainda um gargalo. Outro problema era o fato de o FERUT ser construído com válvulas que queimavam de vez em quando, limitando assim sua confiabilidade, o que fazia retornar ao problema inicial.

⁵ Domínio Público. Cortesia de Valdemar W. Setzer. Disponível em: <http://www.aeitaonline.com.br/wiki/index.php?title=Arquivo:T63-Setzer-com-Zezinho.jpg>

A TCA procurou então ajuda à recém-formada empresa Ferranti-Canadá. A solução proposta pela Ferranti foi o uso de cartões de papel que podiam ser marcados a lápis pelos operadores. Depois eles seriam inseridos em uma máquina que lia as marcas e perfurava no mesmo cartão as posições equivalentes para que o cartão pudesse ser lido por uma leitora comum de cartões. Os dados podiam então ser transmitidos por linha telefônica a um concentrador que faria o processamento de dados vindos de vários operadores ou mesmo de vários escritórios espalhados pelo país.

Além disso, foi proposto construir um computador transistorizado, o que reduziria o problema da confiabilidade baixa causada pela queima de válvulas.

Em 1957 um protótipo do sistema programado no FERUT e ligado a seis terminais, que eles chamavam na época de “transactors”, funcionou perfeitamente.

Em 1959 a TCA executou um projeto de 2 milhões de dólares para construir 350 terminais e um computador chamado Gemini, com duas unidades de processamento: Castor e Pollux. O computador seria construído pela Ferranti-Canada. Esse computador tinha palavras de 25 bits e 4096 (4 KB) palavras de memória principal, depois expandida para 8 KB. A memória secundária era formada por 5 tambores magnéticos, sendo um de reserva, cada um dos quais podia conter 32 KB palavras de 25 bits e seis unidades de fita.

Instalado o sistema na TCA e sendo colocado em uso percebeu-se que os tempos para atendimento de um pedido de reserva baixaram de 90 minutos para 1 segundo! No pior dos casos, quando o sistema estava muito ocupado levava 2 segundos. O sistema ReserVec como um todo conseguia atender 10 requisições por segundo.

A American Airlines (AA), porém, não ficou parada esse tempo todo. Tendo já experiência com o ReserVec, eles também iniciaram o projeto de um sistema de reservas automatizado em 1953. Mas só colocaram seu sistema, o SABRE, em operação em 1964, dois anos depois do ReserVec. Porém, o sistema da AA era mais sofisticado, pois registrava os nomes dos passageiros, enquanto o ReserVec só registrava um código de 3 letras para cada passageiro.

A Ferranti esperava vender seu sistema para outras empresas aéreas, mas nunca conseguiu. As empresas americanas estavam saturadas com máquinas IBM ou UNIVAC, no Canadá não havia outras empresas de porte e na Europa, a Ferranti britânica, em vez de passar a vender o sistema canadense, anunciou que desenvolveria o seu próprio sistema, coisa que a Ferranti-Canadá tomou como insulto. No final o sistema britânico nunca foi desenvolvido e a Ferranti Canada, agora Ferranti-Packard, acabou vendendo a divisão de computadores para se concentrar em outros produtos.

7.9. Supercomputadores Atlas e Titan – 1962

O computador Atlas, quando foi construído em 1962 pela Ferranti, Plessey e Universidade de Manchester, foi considerado um supercomputador (possivelmente o primeiro da história). De fato, ele era de longe o mais poderoso computador de sua época. Dizia-se que quando ele era desligado, a Inglaterra perdia metade de seu poder computacional.

Apenas 3 Atlas foram construídos. Uma versão chamada Atlas II ou Titan, com capacidade para compartilhamento de tempo também teve três unidades construídas. O Atlas foi um projeto de pesquisa realizado pela Universidade de Manchester que visava construir o primeiro processador capaz de processar uma instrução em um milionésimo de segundo, ou um microssegundo.

Ele tinha palavras de 48 bits, 16 KB de memória principal (eletrônica) e 96 KB de memória secundária em tambor magnético. Embora ele tivesse quatro tambores, um sistema de memória virtual permitia que os programadores usassem essa memória e a memória principal como se fosse uma única unidade.

Os Atlas e Titan suportavam os compiladores ALGOL, FORTRAN e COBOL e a maioria de suas unidades foram instaladas em universidades Britânicas, como Manchester, Londres e Cambridge. A **Figura Parte VI: Do Circuito Integrado à Lua-6** é uma foto do Titan de Cambridge.



Figura Parte VI: Do Circuito Integrado à Lua-6: Titan de Cambridge.⁶ 1/1

O conceito de memória virtual do Atlas, ainda hoje usado em praticamente todos os computadores, foi inventado por Tom Kilburn (Reino Unido, 1921-2001). É possível que ele tenha se baseado na tese de doutorado de Fritz-Rudolf Güntsch (Alemanha, 1925-2012), de 1956.

⁶ "Titan computer" by University of Cambridge - http://www.cl.cam.ac.uk/Relics/archive_photos.html. Licensed under CC BY 2.0 via Wikimedia Commons - https://commons.wikimedia.org/wiki/File:Titan_computer.jpg#/media/File:Titan_computer.jpg

Até esse conceito ser inventado, os programadores tinham que dizer explicitamente se estavam acessando dados na memória principal (eletrônica) ou secundária (magnética). Acontece que os computadores só conseguem efetivamente operar os dados que estão na memória principal; um dado que esteja na memória secundária precisa ser carregado para a memória principal e então pode ser operado. Se você tivesse uma memória principal de, digamos 16 KB e uma secundária de, digamos 96 KB, você só poderia trabalhar com no máximo 16 KB de cada vez, e teria que ficar escrevendo e lendo dados na memória secundária. Com a criação da memória virtual, seria como se a memória principal tivesse 96 KB. Os programas poderiam ser escritos considerando uma memória de 96 KB, sem se preocupar em ler e escrever na memória secundária. Um sistema operacional usaria uma técnica chamada de “paginação” para gravar na memória secundária os dados menos usados e trazê-los de volta para a memória principal se eles fossem necessários novamente.

7.10. Mouse – 1962

Dez anos depois da construção da primeira trackball, e possivelmente de forma totalmente independente, Douglas Engelbart (Estados Unidos, 1925-2013) criou o primeiro mecanismo de entrada de dados baseado em uma caixinha que podia ser movida com a mão sobre uma mesa. Como a caixinha tinha um fio que ficava preso na parte de trás e a ligava ao computador, ele batizou o invento como “mouse”, por sua vaga semelhança com esse roedor. Em Portugal, o dispositivo é hoje efetivamente chamado de “rato”.

O mouse foi inventado por Engelbart no Stanford Research Institute (SRI International), que ficou com a patente. Porém, a patente venceu antes que o mouse se tornasse amplamente usado nos computadores pessoais.

Esse primeiro mouse sequer tinha a rollerball em seu interior, mas apenas dois discos ortogonais que giravam em função da fricção com a superfície da mesa. O dispositivo é mostrado na **Figura Parte VI: Do Circuito Integrado à Lua-7**.



Figura Parte VI: Do Circuito Integrado à Lua-7: O primeiro mouse.⁷ 1/2

⁷ "SRI Computer Mouse" by SRI International - SRI International. Licenced under CC BY-SA 3.0 via Wikimedia Commons -

Ele não surgiu do nada, porém, o SRI, com apoio da NASA investigou vários instrumentos que poderiam ser usados para realizar determinadas atividades no computador, como, por exemplo, apontar para um objeto que surgisse numa posição aleatória da tela. Objetos como a caneta de luz, não eram muito ergonômicos, pois era necessário pegá-los da mesa e erguê-los até a tela, num movimento de braço amplo, que dificultava seu uso. Com o mouse sobre a mesa isso era muito mais fácil.

Os primeiros mouses com rollerball interna, foram construídos pela Telefunken a partir de 1968 e eram chamados de “rollkugel”. Esse dispositivo é mostrado na **Figura Parte VI: Do Circuito Integrado à Lua-8.**

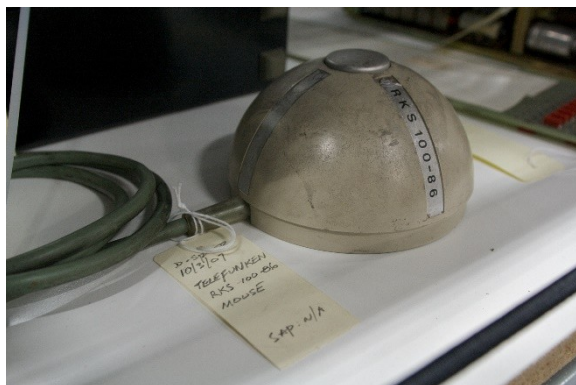


Figura Parte VI: Do Circuito Integrado à Lua-8: Primeiro mouse com rollerball interna.⁸ 1/2

Atualmente os mouses utilizam luz para “sentir” o movimento e não possuem mais a *rollerball* em seu interior. Além disso, a criação dos mouses sem fio eliminou uma de suas principais características de roedor: a cauda.

7.11. SAGE e os Maiores Computadores de todos os Tempos – 1963

O SAGE, ou Semi-Automatic Ground Environment, foi um gigantesco projeto de computação e redes norte-americano, iniciado nos anos 1950 e que ficou operacional a partir de 1961. O SAGE seria uma rede com os maiores computadores construídos em toda a história da humanidade (em volume, não em capacidade de processamento, visto que hoje o sistema sucessor do SAGE funciona em microcomputadores).

O SAGE seria responsável por coletar, transmitir e organizar dados recebidos de vários radares espalhados pelos Estados Unidos e Canadá. Esses dados seriam usados para direcionar o NORAD (North American Aerospace Defense Command) no caso de ataque aéreo. O sistema foi construído principalmente em função da guerra fria, quando se esperava possíveis ataques atômicos soviéticos vindo pelo ártico ou a partir de submarinos.

https://commons.wikimedia.org/wiki/File:SRI_Computer_Mouse.jpg#/media/File:SRI_Computer_Mouse.jpg

⁸ "Telefunken Rollkugel RKS 100-86" by Marcin Wichary - Flickr: Unique mouse from Telefunken, pt. 2. Licensed under CC BY 2.0 via Commons - https://commons.wikimedia.org/wiki/File:Telefunken_Rollkugel_RKS_100-86.jpg#/media/File:Telefunken_Rollkugel_RKS_100-86.jpg

Os computadores gigantescos que formavam a base do SAGE eram modelos IBM AN/F SQ-7⁹ Combat Direction Central, mais conhecidos como “Q7”. Vinte e quatro dessas máquinas foram instaladas nos diversos centros de controle do SAGE. Eram máquinas redundantes, ou seja, cada máquina era formada por dois computadores independentes que processavam os mesmos programas ao mesmo tempo. Se houvesse alguma falha em um deles, eles produziram resultados diversos e assim os técnicos poderiam verificar rapidamente que os resultados não eram confiáveis, evitando guerras acidentais.

As máquinas foram projetadas com base no IBM 701 e usavam memórias de núcleo magnético que seriam versões melhoradas da memória do Whirlwind I.

Cada máquina pesava 250 toneladas e era construída com mais de 60 mil válvulas, gastando cerca de 3 milhões de watts. Como se pode ver na **Figura Parte VI: Do Circuito Integrado à Lua-9**, o console do computador incluía até uma cabine telefônica. A máquina tinha que ser muito rápida, pois deveria receber dados de um número grande de estações de radar e teria que processar estes dados em tempo real.



Figura Parte VI: Do Circuito Integrado à Lua-9: IBM Q7.¹⁰ 1/1

⁹ Army-Navy / Fixed Special eEquipment

¹⁰ "SAGE computer room" by United States Air Force - http://en.wikipedia.org/wiki/Image:SAGE_computer_room.jpg. Licensed under Public Domain via Commons - https://commons.wikimedia.org/wiki/File:SAGE_computer_room.jpg#/media/File:SAGE_computer_room.jpg

No caso de uma invasão de espaço aéreo, o software do SAGE era capaz de prever o melhor ponto de interceptação e automaticamente direcionava mísseis ou aeronaves para este ponto.

Os computadores tinham monitores gráficos de CRT e podiam ser operados através de canetas de luz. O software que rodava no SAGE tinha cerca de 500 mil linhas de código, de longe os maiores programas de computador daquela época.

Um projeto para computadorizar a segurança dos céus norte-americanos já existia desde 1945. Em 1949 foi sugerido por um alto comitê a criação de uma “rede de computadores”. Em 1951 um teste foi feito com a transmissão de dados produzidos por radar através de uma linha telefônica para um computador. Sendo possivelmente a primeira realização deste tipo de transmissão da história. O teste foi conduzido por C. Robert Wieser, do Laboratório Lincoln do MIT, e comprovou a viabilidade do sistema.

O primeiro relatório com os parâmetros iniciais do SAGE foi produzido em 1953 em um projeto conjunto da IBM, MIT e Força Aérea Americana (USAF).

O primeiro protótipo do Q7 foi entregue em 1956 e o primeiro posto de batalha regional do SAGE se tornou operacional em 1958 em Syracuse, New York. O sistema completo ficou operacional em 1963. Porém, quando isso aconteceu, os computadores a válvula já eram considerados obsoletos, já que nessa época a maioria dos novos computadores eram transistorizados ou mesmo implementados com circuitos integrados. Ainda assim, esse parque computacional ficou ativo durante praticamente toda a guerra fria, tendo sido desativado apenas ao longo da década de 1980.

7.12. Sketchpad, o Início da Computação Gráfica – 1963

Em 1963, Ivan Sutherland (Estados Unidos, 1938) defendeu uma tese de doutorado no MIT, onde, orientado por Claude Shannon e apoiado por Marvin Minsky, ele apresentou um sistema chamado “Sketchpad”, um sistema gráfico interativo que permitia ao usuário, usando uma caneta de luz, desenhar pontos em uma tela, mas não só isso, os pontos podiam ser automaticamente ligados por linhas; polígonos podiam ser definidos por desenho livre e seriam corrigidos automaticamente pelo sistema. Ele podia desenhar um círculo a mão livre e o sistema corrigiria as imperfeições, deixando o círculo o mais perfeito possível de acordo com a resolução da tela.



Figura Parte VI: Do Circuito Integrado à Lua-10: Ivan Sutherland operando o Sketchpad.¹¹ 1/1

Mas ainda tinha mais, uma vez que as figuras eram desenhadas elas podiam ser manipuladas como objetos de desenho, ou seja, você podia “pegar” um polígono com a caneta de luz e “arrastá-lo” de um canto da tela para outro. Isso foi uma coisa totalmente inovadora em termos de computação. Esse marco, essa tese, é considerada a fundação da área de Computação Gráfica, e o Sketchpad, um sistema ainda em produção, é considerado o primeiro programa de computação gráfica com interface gráfica.

O Sketchpad teria sido também influenciado pelas ideias de Vannevar Bush, especialmente o Memex. A primeira versão rodava no MIT no computador TX-2, sucessor do TX-0. Um dos fatores que permitiu a Sutherland implementar um sistema interativo e gráfico foi justamente o fato de que o TX-2 implementava o conceito de computação interativa, o mesmo que foi explorado depois pela DEC no seu PDP-1. Com computadores no estilo “batch”, ou seja, computadores que recebiam dados, processavam e depois produziam uma saída, um programa como o Sketchpad seria apenas algo como um conjunto de instruções e dados que o usuário poderia inserir na máquina e, depois, caso o resultado não fosse o que ele queria, teria que corrigir número por número, instrução por instrução, e tentar processar novamente. Mas a era da

¹¹ "SketchpadDissertation-Fig1-2" by Scanned by Kerry Rodden from original photograph by Ivan Sutherland - Electronic edition of Sutherland's Sketchpad dissertation, edited by Blackwell & Rodden. Licensed under CC BY-SA 3.0 via Wikimedia Commons - <https://commons.wikimedia.org/wiki/File:SketchpadDissertation-Fig1-2.tif#/media/File:SketchpadDissertation-Fig1-2.tif>

computação interativa estava começando e assim os usuários de agora em diante poderiam interagir com seus programas a vontade, movendo objetos na tela e fazendo outras coisas que a máquina depois interpretaria e transformaria em dados.

O conceito moderno de programação orientada a objetos também é atribuído a Sutherland. De fato, o termo só passou a ser usado nos anos 1970 após a criação de Smalltalk. Depois, descobriu-se que Simula 67, uma linguagem de programação, já implementava estes conceitos em 1967. Porém, o Sketchpad, de 1963 já utilizava os conceitos de classe, instância e herança, que são fundamentais para a programação orientada a objetos, sendo, portanto, o pioneiro. O trabalho de Sutherland com Alan Kay, seu aluno, levou a influência de suas ideias (tornar o computador acessível ao usuário leigo) para o projeto Dynabook, dentro do qual muitos conceitos da computação moderna foram criados ou consolidados.

O primeiro contato de Sutherland com um computador ocorreu com o kit Simon de 1950, para o qual ele escreveu um programa para dividir dois números. Esse programa, com 8 páginas de fita de papel possivelmente foi um dos maiores, senão o maior, já executado no Simon. Para fazer o programa funcionar, Sutherland precisou implementar um novo circuito para parar a computação dada uma certa condição.

Em 1988 ele recebeu o Prêmio Turing da ACM pela invenção do Sketchpad. Não tanto pelo seu uso, que foi limitado, mas pelas ideias apresentadas na tese. A interface gráfica do Sketchpad influenciou outras interfaces como a da estação de trabalho Xerox Star e finalmente a interface gráfica do Apple Macintosh e Microsoft Windows.

7.13. BASIC – 1964

Até meados dos anos 1960, como já vimos, a maioria dos computadores não eram interativos, mas eram máquinas que recebiam dados, executavam processamento e devolviam uma saída. Não eram máquinas nas quais você pudesse se sentar e, por exemplo, fazer perguntas e obter respostas, ou pesquisar assuntos baseando-se em palavras-chave. As linguagens de programação também tinham essa característica. Você escrevia um programa, ia numa perfuradora de cartões ou fita para obter uma versão que a máquina pudesse ler, compilava o programa, produzindo uma versão em código de máquina, e depois disso estava pronto para o ciclo de entrada de dados, processamento e saída.

O problema é que algumas vezes, o tempo que levava entre você entregar seu programa em cartões perfurados no birô de processamento até obter o relatório de saída impresso podia ser de algumas horas a até um ou mais dias. E se o seu programa tivesse algum erro (bug), ele não ia ser executado, mas você só ficava sabendo isso no dia seguinte.

Essa situação tornava o processo de aprendizagem de programação bastante desgastante para os estudantes. Pensando nisso, John Kemeny (Hungria, 1926-1992) e Thomas E. Kurtz (Estados Unidos, 1928), professores do Dartmouth College, conceberam um sistema de compartilhamento de tempo e uma linguagem de programação simplificada, ambos com propósito didático, ou seja, seria o primeiro ambiente de aprendizagem de programação efetivamente idealizado para este fim.

O sistema de compartilhamento de tempo foi o DTS ou Dartmouth Time-Sharing System, inspirado no sistema do PDP-1. Esse sistema, ficou à disposição dos alunos de Dartmouth, funcionando de 1964 até 1999. Originalmente ele foi implementado em um computador fabricado pela General Electric, o GE-200. A sugestão de que Kurtz fizesse um sistema desse tipo foi dada por John McCarthy, quando Kurtz visitou o PDP-1 no MIT.

Já a linguagem criada por eles foi BASIC (Beginner's All-purpose Symbolic Instruction Code). A linguagem foi inspirada em FORTRAN e ALGOL, mas pretendia ser bem mais simples do que essas duas. Ela podia ser programada a partir de um terminal do GE-200 executando DTS. Não havia editores de texto como os que conhecemos hoje. Os comandos para o DTS e os programas BASIC eram digitados a partir de um editor de uma única linha. Assim, cada linha de um programa BASIC tinha um número que definia sua posição no programa. Normalmente essas linhas eram numeradas de 10 em 10 para facilitar a eventual inserção de outras linhas no meio do código.

Por exemplo, o programador podia digitar a linha 10 e depois as linhas 20 e 30. Se ele resolvesse inserir uma linha entre as linhas 10 e 20 iria escrever no editor de linhas uma linha iniciada, por exemplo, com 15. Assim, uma listagem do programa iria mostrar as linhas 10, 15, 20 e 30.

Se o programador quisesse, por outro lado, apagar uma linha do seu programa, bastaria escrever o número da linha no editor de linhas. Por exemplo, para apagar a linha 20 ele escreveria o editor de linha "20" e ao dar entrada nessa linha a linha 20 do programa seria apagada. Esse você quisesse alterar uma linha de código, precisava escrever novamente toda a linha, iniciando pelo seu número no editor de linhas.

Quando o programador quisesse ver o programa na tela, ele devia executar o comando "LIST" (sem número de linha). Isso fazia com que o programa fosse listado na tela do computador, mas não podia editá-lo diretamente, só através do editor de linhas.

BASIC, desde o início, foi mantida como uma linguagem de domínio público. Devido à sua simplicidade ela rapidamente se espalhou pelas universidades e mesmo empresas. A partir de meados dos anos 1970, com a chegada do microcomputador e a possibilidade de implementar versões do BASIC em máquinas extremamente limitadas, ela chegou a ser a linguagem de programação mais usada no mundo, só perdendo sua liderança com a disseminação das modernas linguagens orientadas a objetos.

O manual do BASIC de 1964 apresenta logo em seu início um exemplo de uso da linguagem. O exemplo é um programa para resolver duas equações lineares simultâneas:

$$A_1X_1 + A_2X_2 = B_1 \quad A_3X_1 + A_4X_2 = B_2$$

Como se trata de apenas duas equações, os valores das incógnitas X_1 e X_2 podem ser calculados por:

$$X_1 = \frac{(B_1 A_4 - B_2 A_2)}{(A_1 A_4 - A_3 A_2)} \quad X_2 = \frac{(A_1 B_2 - A_3 B_1)}{(A_1 A_4 - A_3 A_2)}$$

Porém, por causa da divisão, uma solução única só pode existir se $(A_1 A_4 - A_3 A_2)$ for diferente de zero. Assim, o seguinte programa BASIC resolveria essas equações:

```
10 READ A1, A2, A3, A4
15 LET D = A1*A4-A3*A2
20 IF D=0 THEN 65
30 READ B1, B2
37 LET X1=(B1*A4-B2*A2)/D
42 LET X2=(A1*B2-A3*B1)/D
55 PRINT X1, X2
60 GO TO 30
65 PRINT "NO UNIQUE SOLUTION"
70 DATA 1, 2, 4
80 DATA 2, -7, 5
85 DATA 1, 3, 4, -7
90 END
```

A linha 10 lê os valores 1, 2, 4 e 2 nas linhas 70 e 80, que são linhas que contém os dados. A linha 15 calcula D como a expressão usada no divisor de ambas as fórmulas: $(A_1 A_4 - A_3 A_2)$. A linha 20 verifica se D é zero; se for, pula a execução para a linha 65.

Caso o programa não tenha sido desviado para a linha 65, ele continua na linha 30 que lê os valores -7 e 5 na linha de dados 80. A linha 37 calcula X_1 e a linha 38 calcula X_2 . A linha 55 imprime os dois valores calculados.

A linha 60 envia o programa de volta à linha 30, onde ele vai ler novos valores para B_1 e B_2 , no caso, os valores 1 e 3 da linha 85. As saídas são novamente calculadas e impressas nas linhas 37, 42 e 55.

Uma nova execução da linha 60 leva novamente e volta à linha 30 onde B_1 e B_2 serão carregadas com os valores 4 e -7 obtidos no final da linha 85. O cálculo das saídas é novamente feito e pela terceira vez os resultados são impressos.

A última execução da linha 60 leva o programa à linha 30 onde ele vai parar porque não existem mais dados para serem lidos pelo comando READ.

A existência do comando GO TO (implícito na linha 20) e a falta de estruturas de programação como, por exemplo, recursão fizeram com que o BASIC recebesse muitas críticas, especialmente após o advento da programação estruturada, nos anos 1970.

O BASIC, porém, não foi a primeira linguagem criada pelo grupo de Dartmouth. Eles já haviam proposto a linguagem DARSIMCO (Dartmouth Simplified Code) em 1956, um conjunto de padrões de programação em Assembly que acabou não sendo muito usado pois FORTRAN foi lançada logo no ano seguinte.

Outra experiência ocorreu em 1962, com a linguagem DOPE (Dartmouth Oversimplified Programming Experiment), já com maior influência de FORTRAN e ALGOL, mas ainda uma linguagem muito simples, pois, por exemplo, só permitia uma única operação aritmética por linha de código.

O BASIC criado em Dartmouth era compilado, porém ele se tornou bastante popular, e várias versões interpretadas surgiram. Compilar implica em o computador traduzir o programa em BASIC para um programa na linguagem da máquina, e depois executar este segundo programa. Interpretar implica em o computador ler cada instrução em alto nível e imediatamente executar as instruções correspondentes. O processo de interpretação é usualmente bem mais lento do que a execução de programas compilados, porque durante a execução existe toda a sobrecarga de ler e interpretar os comandos em alto nível antes de executar o código propriamente dito.

Mas, ganhava-se em interatividade. Por exemplo, sistemas BASIC interpretados podiam fazer a checagem sintática dos comandos no momento em que cada linha era digitada. Programas compilados só faziam uma checagem geral após o programa todo estar finalizado. Além disso, os compiladores daquela época eram bem diferentes dos atuais. Kurtz conta que havia uma história sobre um compilador FORTRAN que levava 45 segundos para compilar um programa com uma única instrução “END”. Isso ocorria porque para compilar essa instrução ele tinha que buscá-la em uma tabela de comandos armazenada em uma fita e ela era o último comando da fita.

Várias versões do BASIC foram criadas ao longo da história. Só em Dartmouth, até 1971 foram 6 gerações da linguagem. A proliferação de dialetos fez com que a ANSI (American National Standard Institute) iniciasse em 1974 um projeto para uma especificação mínima padrão de BASIC, o que ocorreu em 1978, e uma especificação padrão surgiu em 1987.

Muitos microcomputadores dos anos 1970 e 1980 vinham de fábrica com uma cópia do BASIC, as vezes até gravada em hardware, como no caso dos Sinclair e assemelhados. Atualmente a versão mais utilizada é o Visual Basic. Segundo a tabela TIOBE¹² em fevereiro de 2016 Visual Basic .NET era a 7ª linguagem de programação mais popular no mundo, e Visual Basic (não .NET) aparece em 12ª.

7.14. IBM 360 – 1964

Pode-se dizer que o início da era da computação moderna ocorreu em 1963 quando John Watson Jr. (Estados Unidos, 1914-1993) anunciou o modelo IBM 360. O 360 é considerado o marco da terceira geração de computadores (computadores com circuitos integrados), inovando em termos de processador e memória, utilizando circuitos integrados capazes de operar instruções em bilionésimos de segundo. Os circuitos integrados usados no 360 foram criados pelo Solid Logic Technology da IBM e eram muito mais rápidos e confiáveis do que outros que existiam. Só para exemplificar, a **Figura Parte VI: Do Circuito Integrado à Lua-11** mostra em primeiro plano o circuito integrado de uma memória sólida de 64 bits sobre uma memória de núcleo magnético, que era o que de melhor se tinha antes disso.

¹² http://www.tiobe.com/index.php/tiobe_index

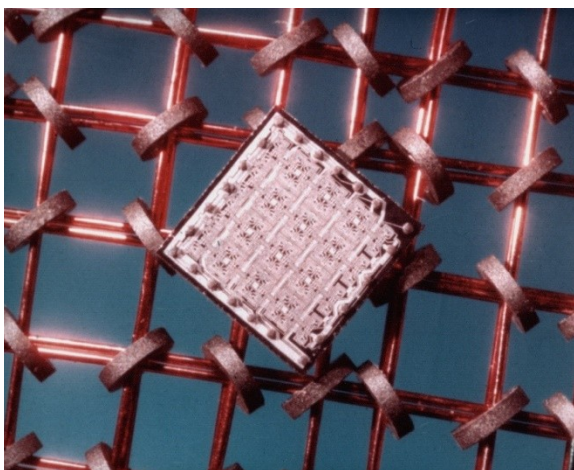


Figura Parte VI: Do Circuito Integrado à Lua-11: Memória sólida do 360 sobre uma memória de núcleo magnético.¹³ 1/2

Porém, a velocidade não foi talvez a principal característica que tornou essa máquina um marco, porque isso era apenas uma evolução. A revolução que este modelo iniciou foi o fato de que com ele passou a existir toda uma família de computadores que eram totalmente compatíveis e atualizáveis, porque teriam o mesmo conjunto de instruções de máquina. Antes disso, quando uma empresa comprava um computador relativamente pequeno e depois atualizava para um mais potente, ela tinha que reescrever boa parte de seus programas, além de comprar novos periféricos como impressoras, unidades de disco e terminais. Com o 360 você podia iniciar com um computador pequeno e ir atualizando seus sistemas na medida da necessidade, sem maior stress ou esforço. O próprio número 360 vinha do fato de que a IBM queria abranger um círculo de 360º de necessidades de usuários. O 360 dava aos usuários a segurança de saber que upgrades seriam tranquilos.

A preocupação com portabilidade era tão grande que emuladores foram construídos para sistemas desenvolvidos em computadores mais antigos. Emuladores eram programas que simulavam uma máquina mas rodavam em outra.

Na época, o projeto do 360 poderia ter sido considerado um suicídio, pois a empresa gastou 5 bilhões de dólares nele e seu sucesso implicaria na obsolescência de muitos outros produtos da empresa.

Seis modelos de processador foram anunciados. O mais rápido era 25 vezes mais veloz que o mais lento. Seis anos depois, novos modelos foram lançados e essa relação passou para 200 para 1.

Além disso, toda uma série de periféricos compatíveis foi construída de forma que uma organização pudesse escolher e utilizar aqueles que fossem necessários na quantidade que precisavam.

¹³ "Electronic Memory" by Steve Jurvetson from Menlo Park, USA - 64-bit Chip. Licenced under CC BY 2.0 via Wikimedia Commons - https://commons.wikimedia.org/wiki/File:Electronic_Memory.jpg#/media/File:Electronic_Memory.jpg

Outro aspecto que o 360 consolidou foi o fim da distinção entre o computador científico e o computador para fins comerciais. Ele podia servir a qualquer uma dessas áreas.

Um IBM 360 modelo 91 (**Figura Parte VI: Do Circuito Integrado à Lua-12**) foi especificamente desenvolvido para a recém criada Agência Espacial Americana, a NASA. Esse computador foi largamente utilizado no projeto Apollo, que levou o homem à Lua.



Figura Parte VI: Do Circuito Integrado à Lua-12: Painel do IBM 360-91 na NASA.¹⁴ 1/1

7.15. Comutação de Pacotes – 1965

Até a invenção da Comutação de Pacotes (Packet Switching) em 1965, a forma de transmitir informação através de redes de telefonia ou assemelhadas consistia em estabelecer uma conexão entre dois pontos, transmitir toda a informação e desligar a conexão. Dessa forma, a banda de comunicação ficava dedicada desde o início até o final a uma mensagem e bloqueada para outras até que ela terminasse. Dessa forma, estimava-se que até 90% do tempo de banda de transmissão era desperdiçado.

Porém, no início dos anos 1960, em plena guerra fria, Paul Baran (Polônia, 1926-2011), trabalhando para a Rand nos Estados Unidos, tentava imaginar uma forma de transportar informação vital entre computadores de forma a resistir a eventuais ataques atômicos. Baran imaginou uma rede de computadores ligada de várias maneiras diferentes, de forma que a destruição de uma linha de comunicação pudesse ser compensada por outras linhas que permaneceriam ativas, mesmo que entre computadores diferentes. Além disso, no modelo que ele imaginou, as informações

¹⁴ "360-91-panel". Licensed under Public Domain via Commons - <https://commons.wikimedia.org/wiki/File:360-91-panel.jpg#/media/File:360-91-panel.jpg>

seriam divididas em pequenos blocos, transmitidos um de cada vez através da rede, seguindo possivelmente caminhos diferentes, para serem reagrupados somente no receptor.

Seria algo como mover uma casa de uma cidade para outra, transportando suas peças, em caminhões diferentes. Os caminhões podem até seguir caminhos diferentes, mas no destino a casa poderá ser novamente montada. No caso da comunicação de informações, isso permitia que diversos computadores se comunicassem com outros, enviando várias mensagens ao mesmo tempo através de vários caminhos. Caminhos que ficassem muito cheios poderiam ser preteridos caso as mensagens, ou partes delas, pudessem transitar por outros caminhos alternativos, como em um sistema de estradas: quando a rodovia principal fica congestionada, os carros começam a procurar vias alternativas. A **Figura Parte VI: Do Circuito Integrado à Lua-13** mostra esquematicamente dois computadores transmitindo 3 mensagens diferentes através de caminhos distintos em uma rede hipotética.

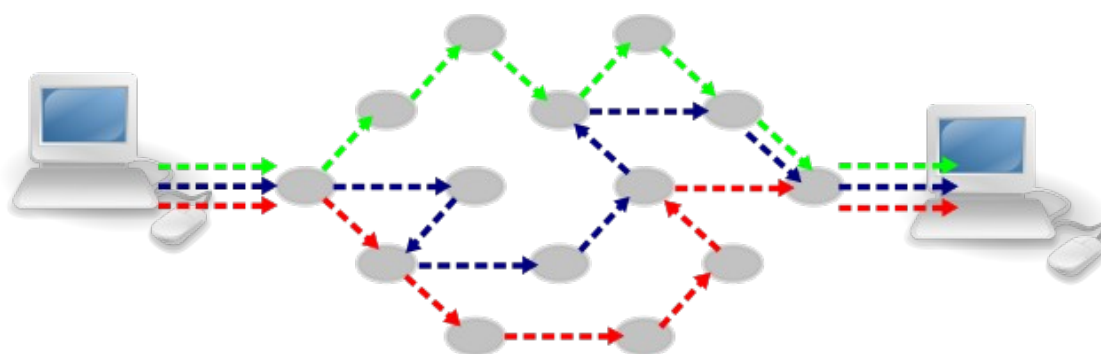


Figura Parte VI: Do Circuito Integrado à Lua-13: Comutação de pacotes.¹⁵ 1/1

Em 1965, Baran obteve aprovação da Força Aérea Americana para testar sua teoria de comutação de blocos. Mas a Agência de Comunicação da Defesa tomou o projeto para si e o complicou tanto que Baran, ao final, não conseguiu testar a ideia e acabou desistindo.

Porém, nessa mesma época, Donald W. Davies (Reino Unido, 1924-2000) na Inglaterra, concebeu um conceito bem similar de forma totalmente independente, que ele chamou de “comutação de pacotes”. Cada pacote de informação teria um tamanho de 128 bytes. Após distribuir um artigo com suas ideias em 1966, ele ficou sabendo da publicação de Baran no ano anterior.

A primeira publicação de Davies sobre o assunto ocorreu em 1967 em um simpósio da ACM. Havia, na época, entre os engenheiros, uma tendência a rejeitar sistemas de comunicação baseados em divisão de mensagens, como a comutação de pacotes, porque essa ideia já era conhecida há várias décadas e sempre tinha sido considerada como economicamente desvantajosa. Assim, ninguém pensava seriamente em retomar o assunto.

¹⁵ "CPT-internet-packetswitching" by Computer-blue.svg: OpenClipartderivative work: Pluke (talk) - Computer-blue.svg. Licenced under CC0 via Wikimedia Commons - <https://commons.wikimedia.org/wiki/File:CPT-internet-packetswitching.svg#/media/File:CPT-internet-packetswitching.svg>

Mas Davies, fazendo uma rigorosa análise econômica, e considerando que a transmissão agora se tratava de dados entre computadores e não mais de voz entre seres humanos, mostrou que neste caso, a ideia de dividir as mensagens em pacotes era extremamente vantajosa.

Hoje, Davies e Baran são considerados os co-inventores desse importante conceito que viabilizou anos depois as redes de computadores.

7.16. CDC 6600 de Seymour Cray – 1965

A CDC, ou Control Data Corporation, foi uma das fabricantes de computadores dos anos 1960 que dividia com outras 6 empresas uma pequena fatia de um mercado que era amplamente dominado pela IBM. Por isso, a IBM e essas outras sete empresas acabaram sendo conhecidas como a “Branca de Neve e os sete anões”.

A CDC foi criada em 1957 por alguns ex-funcionários da Sperry Rand que não estavam satisfeitos com a visão fechada da empresa. A nova empresa iniciou atividades com a fabricação de pequenos periféricos, como memórias de tambor magnético.

Mas em 1958 Seymour Cray (Estados Unidos, 1925-1996) tornou-se engenheiro-chefe da nova empresa e ele tinha grandes ideias. Em 1959 ele já produziu um pequeno protótipo de computador transistorizado de 6 bits conhecido como CDC Little Character. Outros modelos se seguiram rapidamente.

Cray não queria pouco, ele queria produzir o computador mais rápido do mundo, desbancando a IBM dessa posição. Ele conseguiu esse feito em 1965 com o lançamento do CDC 6600 (**Figura Parte VI: Do Circuito Integrado à Lua-14**). Ele ainda não era um computador baseado em circuitos integrados, mas em uma tecnologia de transistores chamada *cordwood* (feixe de lenha) que otimizava muito os tempos de processamento. Além disso, ele acrescentou ao processador principal dez outros processadores para lidarem com entrada e saída e outras funções da máquina de forma independente.



Figura Parte VI: Do Circuito Integrado à Lua-14: CDC 6600.¹⁶ 1/1

O CDC 6600 tinha 400 mil transistores e mais de 160 km de fios. De 1964 a 1969 ele foi o computador mais rápido do mundo, sendo desbancado apenas pelo seu sucessor, o CDC 7600, outro projeto de Cray.

Quando ficou provado que o CDC 6600 era mais rápido do que qualquer outro computador no mundo, Thomas Watson Jr., presidente da IBM ficou furioso. Ele lançou então o que ficou conhecido como o “memorando do faxineiro” no qual questionava como a gigantesca IBM podia ter perdido sua liderança mundial para uma empresa com “34 funcionários, incluindo o faxineiro”.

A IBM rapidamente anunciou modelos muito mais rápidos do Sistema 360, o que freou as vendas do CDC 6600. Porém, a IBM nunca lançou estes modelos e, assim, a CDC respondeu com uma ação judicial antimonopólio. Quatro anos depois a IBM concordou em indenizar a CDC com computadores e dinheiro no valor total de 600 milhões de dólares.

7.17. Touch Screen – 1965

As telas sensíveis ao toque (*touch screen*) que hoje são o padrão de interface para tablets, e smartphones, foram inventadas há meio século, mais precisamente em 1965 na Inglaterra por Eric A. Johnson. Ele descreveu a ideia de usar uma tela capaz de “sentir” o toque humano para operar um sistema de controle de tráfego aéreo, a qual foi publicada na revista *Electronic Letters* sob o título “Touch Display—A novel input/output

¹⁶ "CDC 6600.jc" by Jitze Couperus - Flickr: Supercomputer - The Beginnings. Licenced under CC BY 2.0 via Wikimedia Commons - https://commons.wikimedia.org/wiki/File:CDC_6600.jc.jpg#/media/File:CDC_6600.jc.jpg

device for computers”¹⁷. Em 1967 ele publicou um artigo mais extenso, no qual explicava em maiores detalhes seu dispositivo, bem como apresentava a foto de um protótipo. Johnson recebeu uma patente por essa invenção em 1969.

Em 1972, o engenheiro Bent Stumpe (Dinamarca) foi solicitado a criar um projeto de interface de computador para o CERN (Genebra) que substituísse uma série de botões e interruptores. Stumpe criou um projeto de uma tela sensível ao toque com um número prefixado de botões, a qual se tornou operacional em 1973 e assim permaneceu até 2008.

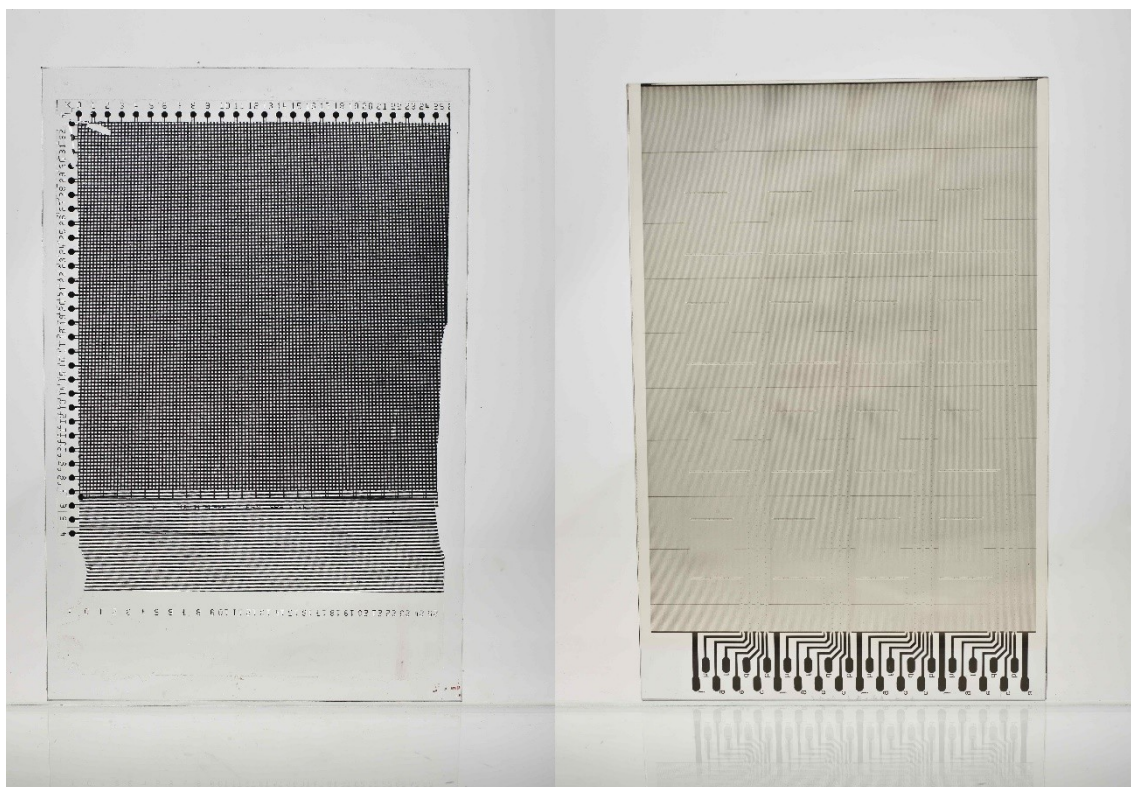


Figura Parte VI: Do Circuito Integrado à Lua-15: Protótipos de tela sensível ao toque desenvolvidos no CERN.¹⁸ 1/1

Diversos modelos de telas sensíveis ao toque foram propostos e construídos nos anos 1970 e 1980. Mas duas tecnologias se tornaram dominantes nos dias atuais: a resistiva e a capacitiva.

Uma tela resistiva é formada por várias camadas, entre as quais estão duas camadas cada uma com minúsculas linhas condutoras que são separadas por pequenos espaços. As camadas são colocadas uma sobre a outra de forma a que uma tenha as linhas na vertical e a outra na horizontal. Elas são separadas entre si por um espaçamento minúsculo feito de tal forma que se um dedo ou apontador colocar uma pequena pressão

¹⁷ Tradução: Tela sensível ao toque – um novo dispositivo de entrada e saída para computadores.

¹⁸ "CERN-Stumpe Capacitance Touchscreen" by Maximilien Brice - <http://www.flickr.com/photos/50410282@N06/4626709556/>. Licensed under CC BY 1.0 via Commons - https://commons.wikimedia.org/wiki/File:CERN-Stumpe_Capacitance_Touchscreen.jpg#/media/File:CERN-Stumpe_Capacitance_Touchscreen.jpg

sobre a camada superior as linhas condutoras nela toquem nas linhas da camada inferior, estabelecendo assim o contato. Uma voltagem elétrica é aplicada a uma camada e sentida pela outra, de forma que se possa passar para o computador as coordenadas nas quais a tela está sendo tocada. Essa tecnologia é usada ainda em alguns dispositivos que demandam alta resistência e baixo custo. Sua desvantagem em tablets é o fato de que ela exige pressão, não apenas o toque do dedo.

A outra tecnologia predominante nos dias atuais é a capacitiva, a qual tem muitas diferentes formas de implementação, mas via-de-regra trata-se de uma superfície condutora sobre uma superfície isolante. Ao colocar o dedo sobre a superfície condutora, como a pele humana conduz eletricidade, haveria uma distorção no potencial eletrostático da tela naquele ponto. Essa distorção poderia ser detectada e transformada em coordenadas, de forma que se saiba qual ponto da tela foi tocado. Essa tecnologia é muito mais sensível do que a resistiva, mas é de implementação mais cara e só funciona com o dedo humano ou materiais condutores. Por exemplo, se você usar uma luva de borracha, não conseguirá nada com uma tela capacitiva.

7.18. Memória DRAM – 1966

Robert H. Dennard (Estados Unidos, 1932) foi contratado em 1958 pela divisão de pesquisa da IBM onde ele intencionava passar uns dois anos para aprender as manhas da indústria, mas onde ele acabou ficando por mais de 40 anos.

Ele estava a cargo de uma pequena equipe que estudava memórias de computador. Na época, um bit de memória precisava de seis transistores para ser mantido. Em 1966, porém, ele teve a ideia de usar um capacitor para segurar o sinal elétrico do bit. O problema com capacitores é que eles vão perdendo a carga elétrica rapidamente se ela não for reforçada. Mas Dennard percebeu que bastaria usar um único transistor para efetuar esse reforço com a velocidade necessária.

Assim, ao invés de usar 6 transistores para representar um único bit de memória, bastaria um transistor e um capacitor. Uma das consequências disso foi um aumento significativo na capacidade de memória dos circuitos integrados, de forma que hoje bilhões de bits podem ser representados em um único circuito.

Essa nova memória baseada em capacitores foi chamada de DRAM, para Dynamic Random Access Memory. O “dynamic” vem do fato de que o capacitor precisa ser reforçado dinamicamente pelo transistor para não perder a carga.

Em 1970, a recém-criada empresa Intel criou um chip (circuito integrado) de número 1103, uma memória DRAM de 1K bit. Houve muita demanda das indústrias por este produto e em 1972 ele se tornou o circuito integrado mais vendido no mundo. Ele é conhecido hoje como o circuito que acabou com a era das memórias de núcleo magnético.

7.19. Chatterbot ELIZA – 1966

Entre 1964 e 1966 o professor Joseph Weizenbaum (Alemanha, 1923-2008) do MIT trabalhou em um programa de computador chamado ELIZA. ELIZA seria o primeiro programa a simular a capacidade e conversação de um ser humano de maneira

semelhante à proposta no Teste de Turing. Assim, ELIZA foi o primeiro *chatbot* da história.

ELIZA era um sistema de casamento de padrões que podia ser carregado com diferentes *scripts*, cada um dos quais corresponderia a um diferente perfil de conversação. O mais famoso destes perfis foi o DOCTOR, que simulava um psicanalista Rogeriano, que trabalharia com a técnica de terapia centrada na pessoa.

ELIZA funcionava no sistema de compartilhamento de tempo MAC do MIT. O sistema consistia de um computador central ligado a um conjunto de terminais que eram basicamente máquinas de escrever elétricas. Cada usuário do computador tinha a impressão de que ele sozinho operava o computador, graças ao sistema de compartilhamento de tempo. O usuário podia datilografar comandos na máquina e o computador escrevia respostas na mesma máquina.

Weizenbaum aproveitou essa característica para implementar o programa ELIZA que “lia” expressões em linguagem natural datilografadas por um usuário e respondia também em linguagem natural, como se fosse um ser humano. Muitas pessoas foram enganadas pelo sistema, pelo menos no início, pois após algum tempo ficava claro que não se tratava de um ser inteligente, mas de uma máquina que simplesmente analisava padrões no texto e produzia respostas baseadas em regras que indicavam como uma determinada pergunta deveria ser convertida em uma determinada resposta.

Abaixo, um exemplo parcial de um dos diálogos de ELIZA. As frases digitadas pelo humano estão em minúsculas e as respostas do computador em maiúsculas. O exemplo foi traduzido diretamente do artigo de Weizenbaum de 1966:

- Os homens são todos iguais.
- DE QUE MANEIRA?
- Eles estão sempre nos incomodando com uma coisa ou outra.
- VOCÊ CONSEGUE PENSAR EM UM EXEMPLO ESPECÍFICO?
- Bem, meu namorado me fez vir aqui.
- SEU NAMORADO FEZ VOCÊ VIR AQUI.
- Ele disse que eu estou deprimida a maior parte do tempo.
- LAMENTO OUVIR QUE VOCÊ ESTÁ DEPRIMIDA.
- ...

O programa funciona de forma muito simples. Ele procura identificar palavras chave no texto, como “deprimida”, no exemplo acima. Ele é capaz de identificar categorias de palavras, por exemplo, X poderia ser uma categoria de palavras negativas como “deprimida”, “cansada”, “triste”, etc. Depois, uma regra diz que ele deve responder a uma frase com a palavra negativa da categoria X usando a frase “lamento ouvir que você está X”. Assim, o computador não precisa saber, e realmente não sabe, o que significa a frase. Mas se depois o humano dissesse “hoje estou cansada” ele responderia “lamento ouvir que você está cansada”.

ELIZA era sofisticado a ponto de ter prioridade entre as regras. Assim, se o programa estivesse lendo um texto e encontrasse uma palavra-chave, ele não daria a resposta

imediatamente, ele continuaria lendo o texto e caso encontrasse outras palavras-chave, ele ao final escolheria a de maior prioridade e aplicaria a regra correspondente a ela somente.

O programa era capaz de prover respostas genéricas caso o humano fugisse do tópico. Por exemplo, se o humano perguntasse “Qual é seu compositor favorito?”, o computador não teria informações para dar uma resposta aceitável a essa pergunta em particular. Assim, ele responderia algo como “Por que essa pergunta lhe interessa?”.

Weizenbaum implementou o sistema na linguagem SLIP (Symmetric List Processor), sua própria versão de uma linguagem funcional semelhante a LISP.

O nome ELIZA foi escolhido em homenagem à personagem Eliza Doolittle, da peça teatral Pygmalion de George Bernard Shaw (Irlanda, 1856-1950). Ela teria sido uma florista que procurou um professor para ter aulas de eloquência e etiqueta e assim poder participar da alta sociedade londrina. Esta peça também teve uma famosa versão musical chamada “My Fair Lady”.

Em 1972 ELIZA teve um encontro histórico com o programa PARRY, um chatterbot que simulava uma pessoa com esquizofrenia. O diálogo, ficou famoso por ter sido a primeira vez que dois computadores conversaram um com o outro usando linguagem humana. O diálogo completo pode ser consultado em <https://tools.ietf.org/html/rfc439>.

7.20. OS/360 – 1967

O Sistema 360 da IBM era mais do que hardware. A empresa lançou em 1964 um ambicioso projeto para desenvolver um sistema operacional, o OS/360. Na verdade, o OS/360 seria uma família de sistemas operacionais totalmente compatíveis e crescentemente mais completos. Para os computadores menores com 8K de memória haveria o BOS/360 (Basic Operating System). Para máquinas médias com unidades de fita, de 16K haveria o TOS (Tape Operating System). Finalmente, para as maiores máquinas com unidades de disco haveria o DOS (Disk Operating System).

A empresa escolheu Frederick J. Brooks Jr. (Estados Unidos, 1931) como líder do projeto. Brooks havia sido um dos mais brilhantes alunos de Robert Aiken, o criador do Harvard Mark I vinte anos antes. O orçamento inicial para o projeto era de 25 milhões de dólares.

Inicialmente, o sistema deveria permitir multiprogramação, ou seja, que vários programas rodassem na máquina ao mesmo tempo, compartilhando fatias de tempo do processador. Mas logo adiaram a ideia quando perceberam a complexidade que isso traria ao projeto.

O projeto iniciou em 1964 com um grupo de 12 designers liderando uma equipe de 60 programadores. Porém, o projeto começou a atrasar devido a pequenos problemas. Em 1965 a equipe foi aumentada para 150 pessoas e o cronograma já estava 6 meses atrasado. Testes foram feitos e se percebeu que o sistema estava muito ruim e precisava de extensiva reprogramação.

Brooks continuava contratando cada vez mais programadores, até chegar a ter uma equipe de 1000 pessoas trabalhando no projeto. O OS/360 foi finalizado e entregue apenas em 1967, um ano atrasado e a um custo de meio bilhão de dólares. Posteriormente, Brooks reconheceu que contratar mais programadores para um projeto que já estava atrasado era exatamente a coisa errada a se fazer. Isso ficou conhecido em engenharia de software como a “Lei de Brooks”. Nas palavras dele: “*The bearing of a child takes nine months, no matter how many women are assigned*”¹⁹. Com isso ele reconhecia uma importante característica da indústria de software: um projeto de software precisa de tempo para amadurecer e organização para funcionar. Posteriormente, em 1975, Brooks publicou um livro que seria um marco para a engenharia de software “The Mythical Man-Month”²⁰ no qual ele apresentava essa e outras ideias.

7.21. MUMPS – 1967

Outra linguagem desenvolvida ainda nos anos 1960 foi MUMPS (Massachusetts General Hospital Utility Multi-Programming System). Ela foi projetada por Neil Pappalardo e outros colegas que trabalhavam no Hospital Geral de Massachusetts em Boston entre 1966 e 1967. A linguagem foi criada especificamente para permitir maior facilidade no tratamento de dados médicos e especialmente dados na forma de texto.

Como o hospital tinha um DEC PDP-7, era natural que a linguagem acabasse tendo a interatividade própria das linguagens interpretadas, diferentemente das compiladas como COBOL e FORTRAN.

MUMPS implementava desde seu início facilidades para trabalhar com estruturas de dados hierárquicas, ou seja, uma maneira de organizar os dados de forma que uma informação possa ter “dependentes”. Assim, por exemplo, o registro referente a uma pessoa poderia ter como dependentes seu contato, histórico médico e dados biométricos. O contato, por sua vez, poderia ter como dependentes o telefone e o endereço, etc. A **Figura Parte VI: Do Circuito Integrado à Lua-16** mostra graficamente esse exemplo de estrutura.

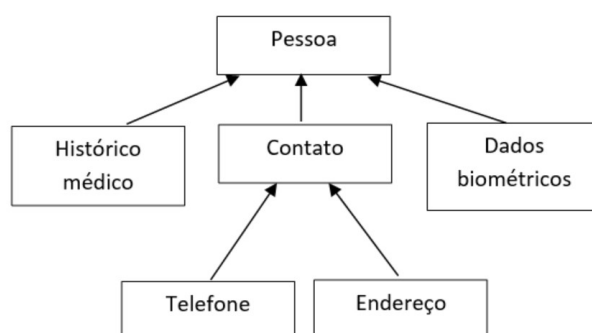


Figura Parte VI: Do Circuito Integrado à Lua-16: Exemplo de estrutura de dados hierárquica.
(TAMANHO: coincidir com fonte do texto)

¹⁹ Tradução: A gestação de uma criança leva 9 meses, não importa a quantas mulheres se atribua a tarefa.

²⁰ Tradução: O mito do homem-mês.

Nos anos 1970 a linguagem MUMPS e seu banco de dados integrado tornaram-se muito populares na comunidade médica. Por ser uma linguagem de domínio público, muitas versões foram implementadas, e a partir de 1972 ela passou por um processo de padronização. O padrão ANSI foi aprovado em 1977 e revisado em 1990 e 1995. Muitas empresas e organizações da área de saúde ainda usam MUMPS para desenvolver seus sistemas nos dias de hoje.

7.22. Simula – 1967

Sketchpad em 1963 já implementava alguns conceitos do que seria posteriormente conhecido como “programação orientada a objetos”. Mas Sketchpad não era uma linguagem de programação, era um programa para desenhar. A programação orientada a objetos surgiu, embora não tivesse ainda esse nome na época, com o projeto de uma linguagem de simulação conhecida como Simula 67.

O projeto de Simula foi iniciado por Kristen Nygaard (Noruega, 1926-2002) da Universidade de Oslo, que escrevia programas de simulação desde 1957. Nygaard tentava achar formas mais adequadas para descrever simulações de eventos discretos. Uma simulação baseada em eventos discretos é um sistema onde se programa o que cada um dos componentes deve fazer em cada instante de tempo. Depois, um motor de simulação vai executar o programa para cada componente repetidas vezes, provocando um comportamento coletivo observável. Por exemplo, se um componente tem seu comportamento definido como mover-se um passo para frente e girar um grau em torno de seu eixo, ao ser colocado sob simulação discreta esse componente vai parecer estar andando em círculos, porque a cada instante de tempo ele anda um passo e gira um grau.

Em 1962 Nygaard percebeu que precisava de alguém com mais conhecimentos em computação e aproximou-se de Ole-Johan Dahl (Noruega, 1931-2002) para lançar ainda este ano a primeira definição de Simula I, uma linguagem de propósito especial assemelhada a ALGOL. Em 1965 a linguagem foi implementada e tornou-se disponível no UNIVAC 1107 e anos depois no Burroughs B5500 e no computador russo URAL-16.

Nygaard e Dalh conseguiram deixar a linguagem relativamente popular na Europa a partir de palestras e cursos, e chegou-se a formar um comitê para sua padronização. Este grupo, formalizou no início de 1968 a versão SIMULA 67, a qual incorporava vários conceitos que mais de uma década depois passaram a ser comuns em linguagens de programação, mas na época não eram.

Eles se inspiraram no conceito de classe definido por Charles A. R. Hoare (Sri Lanka, 1934) e estenderam suas ideias, implementando-as na linguagem. Entre outras ideias que hoje são fundamentais nas linguagens de programação, SIMULA 67 já implementava:

- *Classe e objeto.* A ideia de que uma classe é uma espécie de definição para instâncias ou objetos, que são a realização dessa definição. Por exemplo, “Pessoa” seria uma classe e “João” e “Maria” seriam duas instâncias.

- *Subclasse.* Classes mais abstratas podem ser especializadas em subclasses que compartilham parte da definição. Por exemplo, se a classe “mamífero” é definida como sendo formada por animais que tem pelos e que mamam ao nascer; e se “cão” e “gato” são subclasses de mamífero, então, “cão” e “gato” podem ter suas especificidades, mas eles compartilham a definição de “mamífero”, ou seja, todos os cães e gatos tem pelos e mamam ao nascer.
- *Objetos ativos.* Em Simula qualquer objeto pode funcionar como uma máquina autônoma, ou seja, cada um deles pode operar como se o processador do computador fosse só seu, mesmo que na realidade eles compartilhem fatias de tempo do processador.

A primeira publicação oficial sobre Simula 67 ocorreu em uma working conference sobre linguagens de simulação da IFIP em Oslo em 1967. Simula teve grande influência na definição das modernas linguagens de programação orientadas a objetos. Dahl e Nygaard receberam a Medalha John von Neumann e o Prêmio Turing por suas contribuições à ciência da computação, pela introdução dos conceitos fundamentais de orientação a objetos em linguagens de programação.

7.23. Logo – 1967

Seymour Papert (África do Sul, 1928) é um matemático que trabalhou algum tempo com o epistemologista Jean Piaget (Suíça, 1896-1980) que o influenciou com sua teoria do construtivismo, uma forma inovadora de compreender a aprendizagem como um processo que envolve a ação do sujeito que aprende e não apenas a “transmissão de conhecimento”.

Quando chegou no MIT em meados dos anos 1960, Papert fundou, juntamente com Marvin Minsky (Estados Unidos, 1927), o Laboratório de Inteligência Artificial. Papert trabalhou com a equipe de Wallace Feurzeig (Estados Unidos, 1927-2013) na criação da linguagem Logo. Participaram da equipe também Daniel G. Bobrow (Estados Unidos, 1935) e Cynthia Solomon. Logo não foi criado na universidade, porém, mas em uma empresa de pesquisas de Cambridge chamada BBN ou Bolt, Beranek, and Newman.

A primeira implementação de Logo foi feita em LISP em uma máquina PDP-1. Seu objetivo era desenvolver uma “terra da matemática” na qual as crianças pudessem experimentar com números, palavras, funções e comandos.

Ao contrário da maioria das outras linguagens da época, cujos nomes derivavam de acrônimos ou siglas, Logo teve seu nome inspirado na palavra grega para “pensamento”. Possivelmente, Logo foi a primeira linguagem de programação destinada a desenvolver a habilidade humana de pensar. FORTRAN foi criada para resolver problemas científicos, COBOL para problemas comerciais e Logo foi criada para aprimorar o potencial humano.

Inicialmente, Logo era apenas mais um dialeto de LISP, mas com a introdução do conceito de turtlegraphics seu potencial educacional foi enormemente aumentado. Turtlegraphics é o nome que se dá genericamente a sistemas de desenho baseados em um cursor e geometria radial. Originalmente tratava-se de um pequeno robô que

lembrava o robô programável de Heron de Alexandria, mas com uma caneta que podia desenhar sobre folhas de papel.

O robô obedecia a comandos como andar para frente ou para trás um certo número de “passos” ou girar um certo ângulo para a direita ou para a esquerda. Por exemplo, considerando que o comando para mover a tartaruga para a frente é FD (forward) e que para girar a direita é RT (right), ambos seguidos de um número de passos ou ângulo. O seguinte programa em Logo faria a tartaruga desenhar um quadrado na tela:

```
REPEAT 4 [FD 60 RT 90]
```

Ou seja, a tartaruga deveria repetir quatro vezes a sequência de andar 60 passos e girar 90 graus. O resultado é um quadrado, como mostrado na Figura Parte VI: Do Circuito Integrado à Lua-17

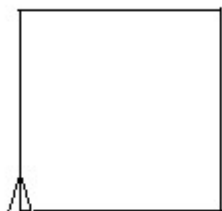


Figura Parte VI: Do Circuito Integrado à Lua-17: Um quadrado desenhado com Logo.²¹ 1/4

O quadrado então poderia ser definido como um novo comando para a linguagem, que é extensível, e usado para formar outras figuras.

A partir então de pouquíssimos conhecimentos em programação é possível começar a experimentar desafios intelectuais. Por exemplo, sabendo-se como desenhar um quadrado podemos inferir como seria o programa para desenhar uma estrela de 5 pontas, como mostrado na Figura Parte VI: Do Circuito Integrado à Lua-18?

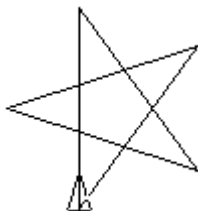


Figura Parte VI: Do Circuito Integrado à Lua-18: Estrela desenhada com Logo.²² 1/4

²¹ "Logo Quadrado" by Leonardo Carneiro de Araujo - Own work. Licensed under CC BY-SA 3.0 via Wikimedia Commons -

https://commons.wikimedia.org/wiki/File:Logo_Quadrado.png#/media/File:Logo_Quadrado.png

²² "Logo estrela5" by Leonardo Araujo - Own work. Licensed under CC BY-SA 3.0 via Wikimedia Commons -

https://commons.wikimedia.org/wiki/File:Logo_estrela5.png#/media/File:Logo_estrela5.png

A resposta seria:

REPEAT 5 [FD 60 RT 144]

O processo de raciocínio a ser desenvolvido seria justamente descobrir qual o ângulo que a tartaruga deve girar para obter o efeito desejado, o que pode ser descoberto com a aplicação de trigonometria simples ou por tentativa e erro. Entre muitas outras possibilidades, uma seria perceber que para desenhar o pentágono a tartaruga precisa dar duas voltas completas sobre o próprio eixo, ou seja, 2×360 graus. Assim, 720 graus dividido por 5 resulta em 5 giros de 144 graus. Por outro lado, girando $360/5$, ou 72 graus, a tartaruga iria desenhar um pentágono e não uma estrela. A vantagem da máquina é que ela dá feedback automático para as tentativas de raciocínio do estudante.

Com a introdução dos microcomputadores nos anos 1970 que implementavam essa linguagem e turtlegraphics, seu uso tornou-se muito popular em escolas no mundo todo. No Brasil uma das maiores incentivadoras dessa tecnologia desde os anos 1980 é Lea da Cruz Fagundes (Brasil, 1930). Psicóloga construtivista, Lea também é uma das pioneiras da área de Informática na Educação do Brasil.

7.24. Forth – 1968

Charles H. Moore (Estados Unidos, 1938) era um programador que não estava satisfeito com as linguagens de programação de alto nível que existiam em 1968. Ele colocou na cabeça que poderia fazer mais, ele queria ser capaz de produzir muito mais programas do que conseguia com FORTRAN. Com este objetivo em mente, ele conseguiu criar uma linguagem que viria a ter características únicas. De certa forma, como explicaremos adiante, ela não é nem compilada nem interpretada, mas alinhavada. Ela produz um código muito eficiente e compacto, que hoje em dia é bastante adequado para dispositivos eletrônicos embutidos ou autônomos como mini robôs. Ela é baseada no uso de uma pilha de dados, o que faz com que as operações sejam *posfixadas*, o que também explicaremos na sequência.

Forth é uma linguagem muito simples; um sistema capaz de rodar Forth pode ser implementado com pouquíssimos bytes. Ao contrário de outras linguagens, não há um compilador que precisa traduzir expressões de alto nível em linguagem de máquina. Forth é constituída por um conjunto inicial de rotinas, cada uma das quais com um endereço de memória definido. Um pequeno programa, o alinhavador de código, faz um loop no qual ele lê em uma posição inicial da memória um valor que deve corresponder ao endereço de uma das rotinas. Então ele desvia o controle para esse endereço fazendo com que a rotina seja executada. Quando ela termina, o controle retorna ao alinhavador que então busca o endereço da rotina seguinte. Assim, um programa Forth consiste em última análise em uma sequência de endereços de memória nos quais estão as rotinas que devem ser executadas.

Porém, um programador Forth não vai usar endereços de memória diretamente em seus programas, ele vai usar “palavras”. Cada palavra em Forth é traduzida em um endereço de memória que o alinhavador vai executar. A associação entre as palavras e endereços de memória é armazenada em uma estrutura chamada dicionário.

Programar em Forth basicamente consiste em criar novas “palavras”, escrevendo sequencias de palavras que formam um procedimento.

Uma característica única de Forth é o fato de que toda a manipulação de dados se dá através de uma pilha. Um número inteiro, por exemplo, consiste em um comando, ou palavra, que tem como efeito colocar o valor representado pelo número no topo da pilha. Assim, um programa como:

```
20 30 50
```

Colocará estes três números na pilha, ficando 50 no topo. Se em seguida o programador escrever a palavra “+”, isso ativará a rotina que soma os dois valores do topo da pilha. Assim, o computador vai somar 30 e 50, colocando o resultado, 80, no topo da pilha, sobre o 20. Se em seguida o programador escrever a palavra “*” o computador vai multiplicar os dois números do topo da pilha, ou seja, 20×80 , o que resulta em 1600.

Assim, o programa completo para calcular $20 \times (30 + 50)$ seria escrito em Forth como:

```
20 30 50 + *
```

Essa forma de escrita, na qual os operadores são colocados depois dos parâmetros é chamada de “notação posfixada”, ou “notação polonesa reversa”.

Leo Brodie publicou um livro interessante sobre Forth em 1987 que ficou famoso por usar caricaturas para exemplificar o funcionamento da linguagem.

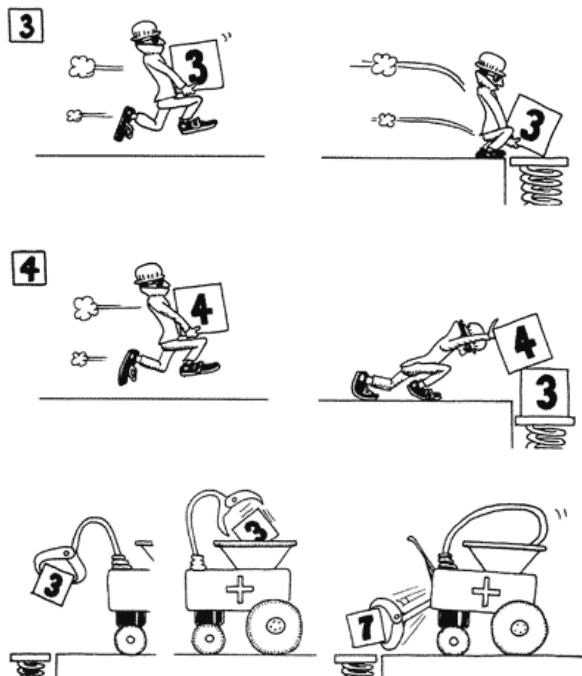


Figura Parte VI: Do Circuito Integrado à Lua-19: Caricaturas sobre Forth de Leo Brodie.²³ 1/2

Um dos desenhos que ficou particularmente famoso foi o dragão do swap, um comando que troca de posição os dois valores mais ao topo da pilha.

²³ © FORTH, Inc. Usada com premissão. Fonte: www.forth.com

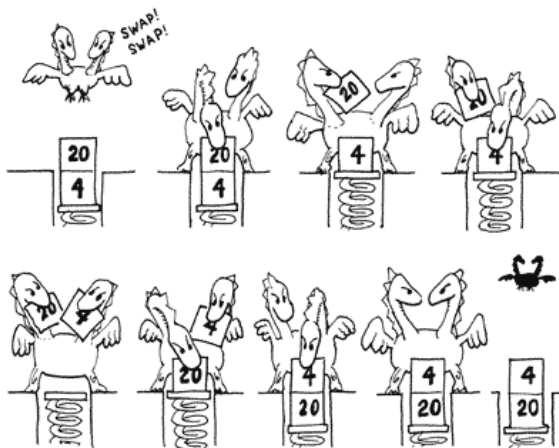


Figura Parte VI: Do Circuito Integrado à Lua-20: Dragão do swap.²⁴ 1/2

A primeira implementação de Forth foi feita por Moore em um IBM 1130. Ele considerava que essa seria uma linguagem de quarta geração e queria que ela se chamasse “Fourth”, mas o 1130 só permitia identificadores com no máximo 5 caracteres. Assim, ele chamou a linguagem de Forth.

7.25. BESM 6, o Computador Soviético – 1968

Sim, a Rússia (União Soviética, na época) também produziu computadores desde cedo. Não ficaram tão conhecidos entre nós, possivelmente devido à Guerra Fria, mas já em 1948 ela começou a construção do MESM (Malaya Elektronno-Schetnaya Mashina²⁵). Ela ficou pronta em 1951, sendo um dos primeiros computadores da Europa.

Logo depois, começou a construção da série BESM (Bolshaya Elektronno-Schetnaya Mashina²⁶). O BESM-1 foi completado em 1952 e possuía 5 mil válvulas. Quando completado era o computador mais rápido da Europa e já usava memória de núcleo magnético.

A partir do BESM-3 os soviéticos passaram a usar transistores. Em 1968 um BESM-4 foi usado para produzir uma das primeiras animações criadas em computador. Um modelo matemático de um gato deu origem a um desenho animado chamado “Kitty” que pode ser assistido em <https://www.youtube.com/watch?v=0O4mm3hXNgA>.

Porém, o mais famoso e importante modelo da série foi o BESM-6 (Figura Parte VI: Do Circuito Integrado à Lua-21), projetado a partir de 1965 e concluído em 1968, um computador transistorizado com processador de 48 bits e capaz de processar um milhão de instruções por segundo (o mais rápido do mundo na época era o CDC 6600, com capacidade de 3 milhões de instruções por segundo).

²⁴ © FORTH, Inc. Usada com permissão. Fonte: www.forth.com

²⁵ Tradução: pequena máquina de computação eletrônica.

²⁶ Tradução: grande máquina de computação eletrônica.



Figura Parte VI: Do Circuito Integrado à Lua-21: BESM-6.²⁷

Mais de 300 unidades do BESM-6 foram fabricadas até 1987, pouco antes do colapso da União Soviética. Ele foi usado extensivamente no programa espacial soviético. O cálculo dos parâmetros orbitais da Soyus levou cerca de 1 minuto para ser processado pelo BESM, enquanto os computadores da NASA precisaram de 30 minutos para fazer os cálculos similares para a Apollo.

7.26. Espada de Dâmocles de Sutherland e o Início da Realidade Virtual – 1968

Ivan Sutherland não sossegou depois de criar o Sketchpad. Em 1968, juntamente com Bob Sproull ele criou um dispositivo que é considerado o primeiro equipamento (capacete) de realidade virtual e realidade aumentada, o Sword of Damocles²⁸.

A animação mostrada no aparelho era muito simples, apenas uma sala modelada com linhas desenhadas. Mas o aparelho conseguia dar a impressão de profundidade, ou terceira dimensão, por mostrar para cada olho uma imagem ligeiramente diferente. Para que o usuário pudesse olhar para diferentes pontos da sala, ele podia mover a cabeça, mas como na época ainda não havia dispositivos sem fio capazes de sentir movimentos, como hoje em dia, eles fixaram o capacete em um suporte de forma que os movimentos giratórios da cabeça do usuário pudessem ser lidos por sensores fixos.

A aparelhagem toda era bem grande e dava a impressão que o usuário estava com uma enorme espada sobre a cabeça. Daí derivou o nome “Espada de Dâmocles” em função da antiga lenda grega de mesmo nome.

O dispositivo era considerado como realidade aumentada porque os visores oculares eram parcialmente transparentes, permitindo ao usuário continuar vendo o mundo real enquanto via também as imagens geradas pelo computador mescladas com ele.

7.27. Oki Wiredot, a Primeira Impressora Matricial – 1968

As primeiras impressoras de computadores eram basicamente máquinas de escrever modificadas. Nestas máquinas, cada letra ou símbolo era gravada em alto relevo em um

²⁷ By Sergey Vakulenko - <http://ramlammammambam.livejournal.com/110695.html>, with the agreement from the author (on the same page, in the post comments)., CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=7914366>

²⁸ Tradução: Espada de Dâmocles.

martelo, o qual, batido contra uma folha ou fita de carbono superposta a uma folha de papel produzia o desenho da letra no papel.

Mas em 1968 a fabricante japonesa OKI, criou um modelo diferente de impressora, a impressora matricial OKI Wiredot.

Era ainda uma impressora de impacto, portanto diferente das jato-de-tinta ou laser. Mas em vez de ter um martelo específico para cada letra ela tinha um conjunto de agulhas que se movia sobre o papel. A cada instante algumas agulhas batiam no papel e outras não. O movimento combinado com as batidas produzia no papel o desenho de qualquer letra ou símbolo que se quisesse, bem como desenhos complexos. A Figura Parte VI: Do Circuito Integrado à Lua-22 mostra um exemplo de impressão obtida com essa tecnologia.

ystem where a
ld allow us t
mercial supplier.



Figura Parte VI: Do Circuito Integrado à Lua-22: Exemplo de impressão matricial.²⁹ 1/2

As matriciais tornaram-se o padrão da indústria de impressoras a partir dos anos 1970 até serem suplantadas pelas jato-de-tinta e laser; mas como estas últimas não são baseadas em impacto, elas não são adequadas para imprimir várias cópias carbono de uma vez. Assim, em aplicações que exigem cópias carbono, ou impressão baseada em impacto por qualquer outro motivo, as matriciais ainda continuam sendo usadas.

Há basicamente dois tipos de impressoras matriciais, as de coluna e as de linha. As impressoras de coluna têm uma fileira vertical de agulhas que corre horizontalmente com um cabeçote imprimindo assim caractere por caractere em uma linha. Elas são usualmente mais lentas, com capacidade de no máximo 500 caracteres por segundo.

Já as impressoras de linha, tem uma ou mais fileiras horizontais de agulhas com largura equivalente à largura do papel. Assim elas literalmente imprimem uma linha inteira de cada vez, sendo bem mais rápidas, com velocidades que excedem facilmente 1000 caracteres por segundo.

7.28. HP 9100A, a Primeira Calculadora Programável Pessoal – 1968

Em 1968 a Hewlett-Packard criou a HP 9100A, considerada a primeira calculadora programável de uso pessoal. Na verdade, na época ela foi responsável por cunhar pela primeira vez o termo “computador pessoal”, isso vários anos antes do Altair e Xerox Alto. Porém, por questões de marketing, a HP não quis que ela fosse chamada de computador, mas de calculadora. Isso porque na época computadores era máquinas enormes e para comprá-las as empresas tinham processos complexos que envolviam os diretores de mais alto escalão. Porém, qualquer gerente médio podia comprar calculadoras.

²⁹ "Dot matrix example text" by Fourfour - Own work. Licensed under CC BY-SA 2.5 via Commons - https://commons.wikimedia.org/wiki/File:Dot_matrix_example_text.png#/media/File:Dot_matrix_example_text.png



Figura Parte VI: Do Circuito Integrado à Lua-23: HP 9100A.³⁰

Mesmo assim, o preço de uma HP não era acessível para a maioria dos mortais: cerca de 5 mil dólares, equivalendo a cerca de 34 mil dólares em 2015.

O modelo 9100A inaugurou a tradição da HP na construção de calculadoras científicas programáveis e também o uso da notação polonesa reversa em suas linguagens de programação.

7.29. Dynabook de Alan Kay – 1968

Alan C. Kay (Estados Unidos, 1940) foi aluno de Ivan Sutherland, em meados dos anos 1960, com o qual conheceu o Sketchpad e seus conceitos subjacentes. A partir de 1967 ele começou a trabalhar em um conceito de máquina programável por usuários, inclusive crianças, que ele chamou de máquina FLEX. Seu encontro com Seymour Papert em 1968 e sua introdução aos conceitos relacionados a Logo motivou-o a continuar investindo nessa direção. Outra influência importante foi Vannevar Bush e seu Memex.

Em 1970 a Xerox, ciente de que seu modelo de negócio baseado em cópias de papel estaria com os dias contados na medida em que os computadores fossem cada vez mais usados para armazenar e consultar informação resolveu investir em pesquisas para tentar desvendar e antecipar esse futuro para ela própria. Foi assim fundado o centro de pesquisas Xerox Parc em Palo Alto, CA, que ficou famoso por várias invenções

³⁰ "HP 9100A Calculator 1968 front" by Swtpc6800 en:User:Swtpc6800 Michael Holley - Own work. Licenced under Public Domain via Wikimedia Commons - https://commons.wikimedia.org/wiki/File:HP_9100A_Calculator_1968_front.jpg#/media/File:HP_9100A_Calculator_1968_front.jpg

relacionadas à computação nos anos 1970. Kay foi contratado como um dos pesquisadores deste centro e logo aliou-se ao projeto Smalltalk, que visava desenvolver uma linguagem de programação acessível ao público infanto-juvenil. Seus conhecimentos sobre orientação a objetos oriundos do Sketchpad o levaram a trabalhar para que Smalltalk fosse efetivamente uma linguagem orientada a objetos.

Tanto Kay quanto o Parc foram extremamente prolíficos em gerar novas ideias, mas a que vamos descrever aqui é uma concepção que Kay teve em 1968, o Dynabook, que seria um computador pessoal para crianças de todas as idades. O conceito foi apresentado em um artigo de 1972 no qual ele enfatizava que uma máquina como o Dynabook seria importante para permitir a aprendizagem construtivista, melhorando assim o sistema educacional da época.

No início do artigo ele mostra duas crianças sentadas na grama com Dynabooks jogando Spacewar! em rede e aprendendo sobre gravidade e movimento uniforme enquanto se divertem.

O Dynabook seria muito semelhante aos nossos modernos notebooks ou tablets. Deveria pesar menos de um quilo, ter o tamanho de um livro, com uma tela e um teclado portáteis e interface gráfica com o usuário, semelhante ao Sketchpad. A **Figura Parte VI: Do Circuito Integrado à Lua-24** mostra Kay e seu protótipo. Ele deveria também poder conversar com outros computadores através da linha telefônica, antecipando os modems construídos mais tarde.

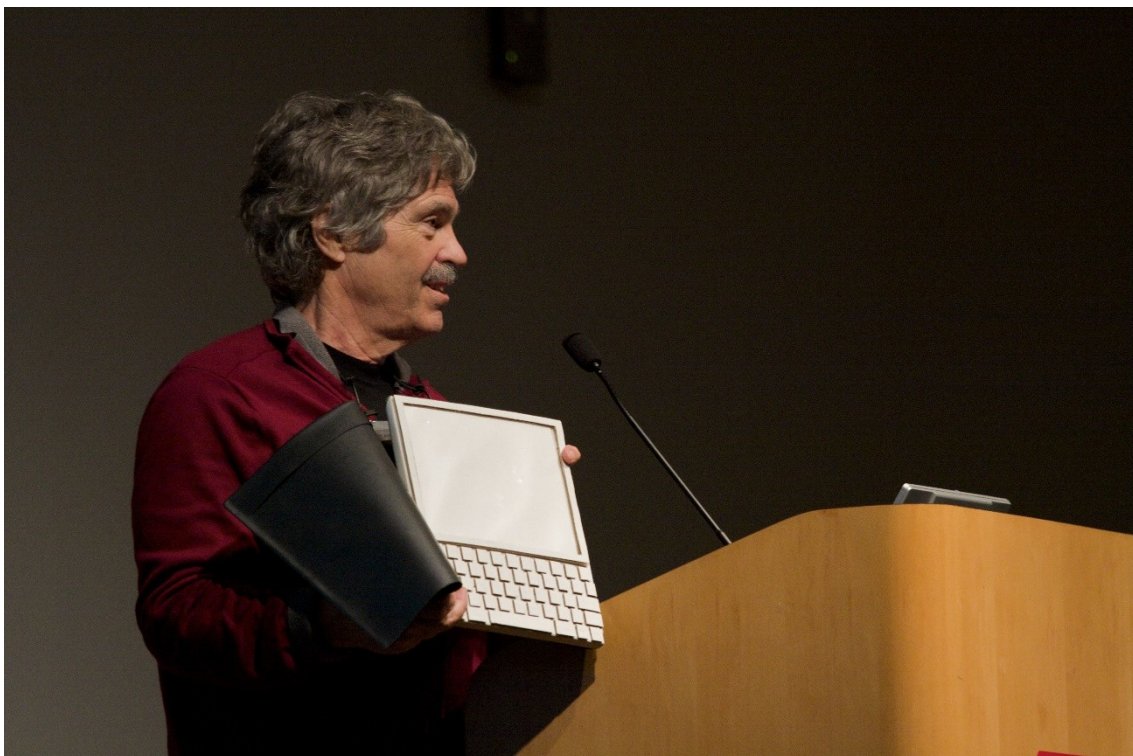


Figura Parte VI: Do Circuito Integrado à Lua-24: Alan Kay segurando um protótipo do Dynabook.³¹

³¹ "Alan Kay and the prototype of Dynabook, pt. 5 (3010032738)" by Marcin Wichary from San Francisco, U.S.A. - Alan Kay and the prototype of Dynabook, pt. 5. Licenced under CC BY 2.0 via Wikimedia Commons -

Com o Dynabook seria possível “instanciar” (baixar) livros na máquina, ao invés de comprá-los em papel. A tela teria que ser de cristal líquido, ele sugere, porque consumiria menos energia, já que o aparelho seria mantido por baterias. O teclado não deveria ter partes móveis, mas ser sensível ao toque e produzindo um feedback sonoro quando tocado. O preço do Dynabook deveria ser equivalente ao de uma TV, cerca de 500 dólares na época, uma previsão que surpreendentemente se realizou nos dias de hoje visto que os preços de televisores e notebooks ou tablets se equiparam mais ou menos neste patamar.

O Dynabook nunca foi construído porque estava muito à frente de seu tempo. Mas ele inspirou um dos primeiros computadores pessoais, o Xerox Alto, além das interfaces gráficas dos computadores Apple e do sistema operacional Windows.

7.30. A Arte de Programação de Computadores de Knuth – 1968

Possivelmente o maior e mais importante tratado sobre programação de computadores é de autoria de Donald Knuth (Estados Unidos, 1938). O trabalho começou a ser publicado em 1968 e ainda não foi terminado.

Knuth iniciou o projeto em 1962, inicialmente pensado como um único livro de 12 capítulos. Ele se motivou a isso quando começou a escrever sobre programação de compiladores e percebeu que poderia ir bem mais além, produzindo um tratado completo e detalhado sobre técnicas de programação e análise de algoritmos.

Ele finalizou o primeiro manuscrito em 1966, com 3000 páginas, assumindo que a transcrição para a versão tipográfica seria da ordem de 5 para 1. Porém, na verdade a ordem era de 1,5 para 1, e assim o livro finalizado ficaria com cerca de 2000 páginas, o que era muita coisa, especialmente considerando que ele era recém-formado; nenhuma editora aceitaria esse risco. Porém, com apoio e entusiasmo de outros pesquisadores ele conseguiu aprovar a produção de um projeto de 7 volumes, cada um dos quais contendo um ou dois capítulos do projeto original. A organização dos volumes ficou assim:

- Volume 1 – algoritmos fundamentais
 - Capítulo 1 – conceitos básicos
 - Capítulo 2 – estruturas de informação
- Volume 2 – algoritmos seminuméricos
 - Capítulo 3 – números aleatórios
 - Capítulo 4 – aritmética
- Volume 3 – ordenação e pesquisa
 - Capítulo 5 – ordenação
 - Capítulo 6 – pesquisa
- Volume 4 – algoritmos combinatórios (parcialmente publicado)
 - Capítulo 7 – pesquisa combinatória

- Capítulo 8 – recursão
- Volume 5 – algoritmos sintáticos (lançamento estimado para 2020)
 - Capítulo 9 – escaneamento léxico, pesquisa em texto e compressão de dados.
 - Capítulo 10 – técnicas de parsing.
- Volume 6 – Teoria das linguagens livres de contexto (planejado)
- Volume 7 – Técnicas de compilação (planejado)

Knuth ofereceu, desde o início um dólar hexadecimal (2,56 dólares) para qualquer pessoa que encontrasse um erro em um de seus livros. As revisões de seu trabalho, assim, contribuíram para sua popularidade e alta precisão, sendo hoje um dos trabalhos mais respeitados e influentes na área de algoritmos. A **Figura Parte VI: Do Circuito Integrado à Lua-25** mostra alguns volumes publicados.



Figura Parte VI: Do Circuito Integrado à Lua-25: The Art of Computer Programming.³²

No livro há vários níveis de dificuldade nos exercícios, desde exercícios de aquecimento até problemas de pesquisa até hoje não resolvidos. Bill Gates teria afirmado certa vez que alguém que conseguisse ler os livros de Knuth e entender tudo, poderia mandar o currículo para ele para ser imediatamente contratado.

7.31. Crise do Software – 1968

Em 1968 a OTAN organizou a primeira conferência de Engenharia de Software em Garmish na Alemanha. Nessa conferência, alguns participantes cunharam e passaram a divulgar o termo “crise do software”, que foi por décadas uma das questões mais comentadas dentro da área de engenharia de software. Basicamente, se constatou já naquele tempo que os computadores tinham ficado muito poderosos e que, pela falta de técnicas e processos adequados, a produção de software não acompanhava esse crescimento.

Basicamente projetos de software custavam mais do que o previsto e passavam do prazo, os produtos tinham qualidade insatisfatória e eram difíceis de manter; isso quando os projetos não eram simplesmente cancelados.

O termo foi bastante divulgado por Edsger Dijkstra (Holanda, 1930-2002), em um artigo publicado em 1972. Ele se intitulava como o primeiro programador da Holanda, pois iniciou essa atividade em 1952 com o EDSAC em Cambridge, e ninguém outro nunca questionou esse título. Naqueles primeiros anos, ele recorda que invejava seus colegas que trabalhavam com hardware porque eles sabiam exatamente o que faziam enquanto ele, programador, não era nem reconhecido como profissional. Quando se casou ele quis colocar como profissão “programador”, mas a prefeitura não permitiu e assim, em seu documento de casamento acabou constando a profissão de “físico teórico”.

Em relação à evolução da computação nos anos 1950 e 1960 ele dizia que os mecanismos de interrupção introduzidos nas máquinas mais modernas da época tinham criado comportamentos imprevisíveis. Além disso, ele comentava que quando os primeiros computadores foram construídos, programação era um problema menor, mas no final dos anos 1960 já havia se tornado extremamente difícil, considerando a capacidade das novas máquinas. A coisa teria piorado muito com o surgimento dos computadores de 3ª geração, como o IBM 360. Ele lamentava que, com o lançamento de tais máquinas, os programadores esperavam que suas especificações fossem claras e sistemáticas, mas o que se observou foi bem o contrário: as especificações eram complexas e obscuras, o que fazia dos programadores as mais infelizes criaturas da terra.

Os anos 1970 ficaram conhecidos para a engenharia e software como os anos da crise do software. Já os anos 1980 foram os anos das “balas de prata”, nos quais vários metodologistas apresentavam soluções para o problema de produção de software que alegadamente resolveriam todas as dificuldades. Mas apenas na virada do Século XXI chegou-se a era da maturidade em que se percebeu que diferentes tipos de sistema exigem diferentes tipos de processos, não havendo assim uma “receita de bolo” definitiva, embora muito se tenha evoluído hoje nessa área.

7.32. Microprocessador – 1968

Com a invenção dos circuitos integrados em 1960, logo várias empresas começaram a imaginar a possibilidade de integrar em uma única peça todos os circuitos necessários para uma unidade central de processamento (CPU) de um computador. Tal circuito seria o que hoje chamamos de microprocessador: um computador completo em um único chip de silício. Isso foi conseguido por várias dessas empresas de forma independente a partir de 1968.

Oficialmente, a invenção do microprocessador é atribuída a Marcian E. “Ted” Hoff (Estados Unidos, 1937). Logo após terminar seu doutorado, Hoff resolveu trabalhar na indústria. Ele conseguiu uma entrevista de emprego com Robert Noyce, o inventor do chip de silício, que saiu da Fairchild para ajudar a fundar uma *startup* chamada Intel. Noyce perguntou o que Hoff imaginava que seria a nova geração de chips de silício e Hoff disse “memória”. Como era exatamente o que Noyce esperava ouvir, ele foi contratado e tornou-se o empregado número 12 da Intel.

Hoff foi o primeiro a perceber que com a nova tecnologia de circuitos de silício da Intel e uma arquitetura simplificada seria possível juntar em um único chip os vários circuitos

necessários para construir um microprocessador. O projeto originou-se da demanda de uma fabricante de calculadoras japonesa, a Busicom, por um conjunto de circuitos de alta performance. Hoff propôs o uso de uma memória estática (ROM) com um programa armazenado, ligada a uma memória dinâmica (RAM) para dados, um processador de entrada e saída (I/O) e um microprocessador de 4 bits. Mas na época ele ainda não tinha o conhecimento necessário para projetar este circuito e o projeto ficou só na ideia até o lançamento do Intel 4004 três anos depois.

Porém, em 1990 Gilbert Hyatt, após um processo de 20 anos contra o escritório de patentes, foi reconhecido como inventor do microprocessador também em 1968, alguns meses antes da invenção da Intel. Porém, a patente foi novamente invalidada posteriormente.

Praticamente na mesma época, a Texas Instruments também estava criando seu microprocessador, o TMS 1000, que ficou pronto em 1971 e passou a ser comercializado em 1974.

Também em 1968, outra empresa, a Garrett AiResearch, produziu um microprocessador para o controle de voo dos novos caças F-14 Tomcat. O projeto, completado em 1970, foi considerado tão avançado que a Marinha se recusou a permitir sua publicação até 1997, de forma que ele acabou sendo muito pouco conhecido.

7.33. MULTICS – 1969

A experiência do MIT com compartilhamento de tempo não terminou com o CTSS. Três anos após a criação do CTSS eles iniciaram o projeto de um sistema operacional comercial com compartilhamento de tempo, o MULTICS (Multiplexed Information and Computing Service).

O projeto foi liderado por Fernando J. Corbató (Estados Unidos, 1926). O projeto, iniciado em 1964 era uma cooperação do MIT, General Electric e Bell Labs. O plano foi apresentado em 1965 em uma série de seis artigos. A partir de 1969 o serviço se tornou operacional no campus do MIT, servindo a milhares de estudantes e professores.

A Bell saiu do projeto em 1969, mas algumas das pessoas que lá trabalhavam para o projeto MULTICS acabaram depois ajudando a criar o Unix. Em 1970 a GE vendeu seus negócios relacionados a computação, inclusive o MULTICS, para Honeywell, hoje Bull. Conta-se que a IBM não quis participar do projeto, embora procurada pelo MIT, porque estava para lançar o OS/360 e não via uso para as ideias do MULTICS.

Depois que a Bull parou de dar suporte ao MULTICS, seus usuários gradualmente migraram para sistemas UNIX. A última instalação de MULTICS que se tem conhecimento foi encerrada no ano 2000 no Canadá.

O MULTICS incorporava os requisitos para oferecimento de computação como serviço, ou seja, você poderia comprar poder e tempo de computação como compra água ou eletricidade. O sistema era altamente modular e novos módulos podiam ser adicionados conforme a necessidade.

Entre outras características, MULTICS se destacava pelo uso de memória virtual. Múltiplos processadores poderiam compartilhar uma mesma memória física. Ele implementava várias linguagens de programação: PL/I, BCPL, BASIC, APL, FORTRAN, LISP, SNOBOL, C, COBOL, ALGOL 68 e Pascal. Além disso, programas em uma das linguagens poderiam chamar rotinas escritas nas outras linguagens.

MULTICS implementou o primeiro sistema comercial de banco de dados relacional, o MULTICS Relational Data Store (MRDS). Além disso, ele foi o primeiro sistema operacional a receber a certificação de segurança nível B2 pelo National Computer Security Center do governo americano, fato que é celebrado no botão mostrado na **Figura Parte VI: Do Circuito Integrado à Lua-26**.



Figura Parte VI: Do Circuito Integrado à Lua-26: Botão celebrando a certificação de segurança B2 obtida pelo MULTICS.³³ 1/4

Além disso, ele foi inteiramente implementado em uma linguagem de alto nível, a PL/I, da IBM. Até então sistemas operacionais eram implementados em linguagem de máquina; implementá-los em linguagem de alto nível era considerado muito arriscado, pois o código resultante poderia não ser eficiente. A realidade, porém, mostrou que os compiladores eram capazes de gerar código muito mais eficiente e em um tempo muito mais curto do que programadores humanos com linguagem de máquina eram capazes de fazer.

7.34. ARPANET – 1969

A ARPANET pode ser considerada, sem sombra de dúvida, como a precursora mais antiga da atual Internet. A motivação para sua criação muitas vezes é confundida com a criação da comutação de pacotes por Davies e Baran, e por esse motivo, muitas vezes afirma-se que a ARPANET foi criada como uma rede de computadores para resistir a um ataque nuclear. Mas segundo seus criadores, isso não é verdade; a motivação para criar uma rede deste tipo estava no fato de que os meios de comunicação na época eram tão precários que as conexões nem precisavam de ataques nucleares para falhar – elas faziam isso espontaneamente.

Inclusive conta-se que a primeira mensagem transmitida pela ARPANET foi “LO”. Um operador estava tentando transmitir a mensagem “LOGIN”, mas após digitar a segunda letra a rede caiu devido a um erro de programação e só foi restabelecida mais de uma hora depois. Assim, a história registrou “LO” como a primeira mensagem transmitida pela ARPANET e consequentemente pela Internet.

³³ Cortesia de Tom Van Vleck. Disponível em: <http://www.multicians.org/multics-fer.html>

ARPANET ou Advanced Research Projects Agency Network foi uma rede de comutação de pacotes criada pelo Departamento de Defesa norte americano. Seu projeto foi inspirado por Joseph C. R. Licklider (Estados Unidos, 1915-1990). Curiosamente, ele batizou a rede, em um memorando de 1963 como “Intergalactic Computer Network”³⁴; ela seria o principal e essencial meio de comunicação entre governos, instituições e indivíduos.

Ainda em 1963 ele convenceu Ivan Sutherland e Bob Taylor de que o conceito era importante. Taylor, especificamente tinha 3 terminais de computador em seu escritório e eles não se comunicavam entre si. Ele percebeu que seria melhor ter um único terminal que o conectasse a qualquer computador que ele quisesse.

Em 1968 Taylor havia preparado um plano completo para a rede de computadores e a licitação para construir essa rede foi vencida pela BBN (Bolt, Beranek, and Newman) em abril de 1969. A rede proposta por Taylor e rapidamente construída era composta por pequenos computadores chamados de IMP (Interface Message Processor), equivalentes aos atuais roteadores. O primeiro destes aparelhos, instalado na UCLA é mostrado na **Figura Parte VI: Do Circuito Integrado à Lua-27**. Esses computadores funcionavam como portas de passagem (*gateways*) conectando recursos locais à rede. Inicialmente as conexões eram formadas por linhas telefônicas dedicadas com taxas de comunicação inicial de 56 mil bits por segundo (56 Kbit/s). O sistema todo foi instalado em 9 meses.

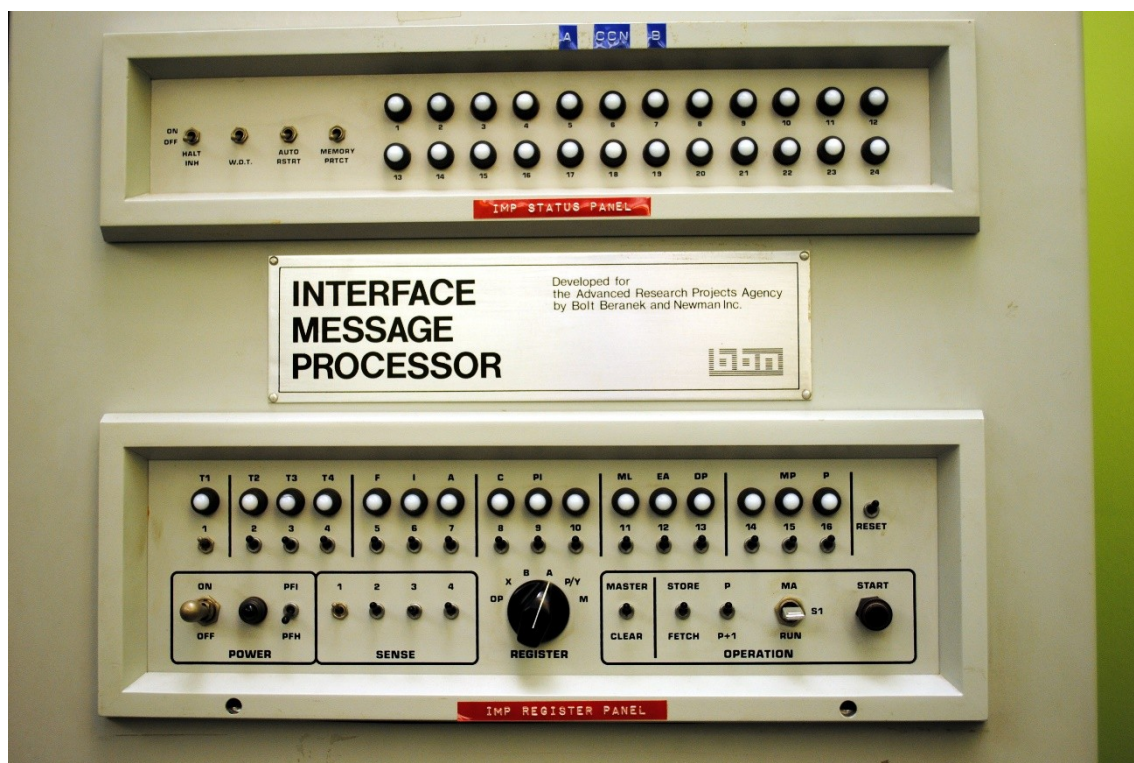


Figura Parte VI: Do Circuito Integrado à Lua-27: O IMP da UCLA.³⁵ 1/1

³⁴ Tradução: Rede de computadores intergaláctica.

³⁵ "Interface Message Processor Front Panel" by FastLizard4 - Own work. Licenced under CC BY-SA 3.0 via Wikimedia Commons -

Inicialmente, a ARPANET consistia de apenas quatro IMPs:

- UCLA (University of California at Los Angeles) com um computador SDS Sigma 7.
- SRI (Stanford Research Institute) com um SDS 940.
- UCSB (University of California at Santa Barbara) com um IBM 360/75.
- Univesity of Utah, com um DEC PDP-10.

A primeira mensagem, mencionada acima, "LO", foi enviada por um estudante da UCLA chamado Charley Kline no dia 29 de outubro de 1969. O primeiro link permanente foi estabelecido em 21 de novembro entre a UCLA e o SRI e os quatro IMP estavam online em dezembro do mesmo ano.

A partir desse embrião, a rede se expandiu com novas universidades, centros de pesquisa e órgãos do governo sendo incorporados. A **Figura Parte VI: Do Circuito Integrado à Lua-28** mostra um esquema de como estava a ARPANET oito anos após sua criação, em 1977.

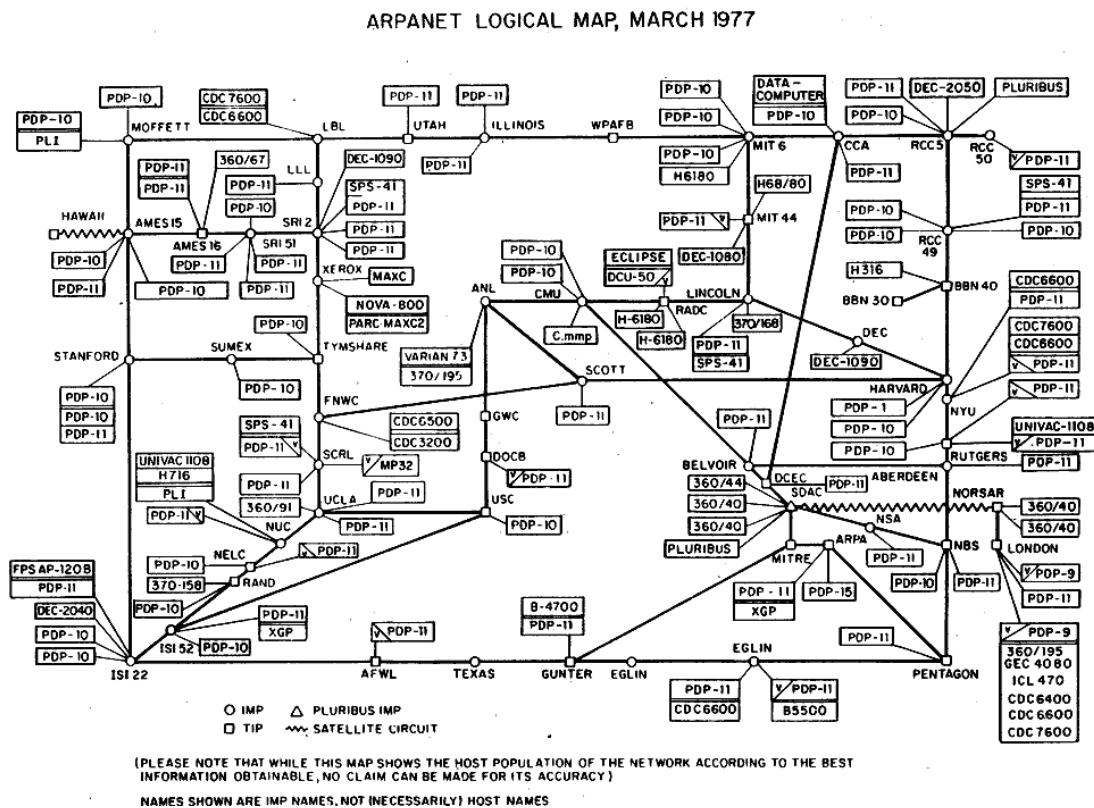


Figura Parte VI: Do Circuito Integrado à Lua-28: Esquema da Arpanet por volta de 1977.³⁶ 1/1

https://commons.wikimedia.org/wiki/File:Interface_Message_Processor_Front_Panel.jpg#/media/File:Interface_Message_Processor_Front_Panel.jpg

³⁶ "Arpanet logical map, march 1977" by ARPANET - The Computer History Museum ([1]), en:File:Arpnet-map-march-1977.png. Licensed under Public Domain via Commons - https://commons.wikimedia.org/wiki/File:Arpanet_logical_map,_march_1977.png#/media/File:Arpanet_logical_map,_march_1977.png

Em 1973 ela se liga à Noruega, o primeiro país, fora os Estados Unidos, a integrar a rede. Em 1975 ela foi declarada como operacional e seu controle foi assumido pela Agência de Comunicações de Defesa, que a reestruturou separando os nodos militares na MILNET (Military Network). A partir de 1983 o TCP/IP tornou-se o principal protocolo de comunicação da ARPANET e ela própria tornou-se uma subrede da nova Internet. Mas isso é história para mais adiante. A ARPANET foi oficialmente aposentada em 1990, com direito a um Réquiem (missa fúnebre).

7.35. Arkay CT-650, o Computador de Clipe de Papel – 1969

Irving Becker, em 1969 inspirou-se em um livro de 1967 para construir um modelo de computador pessoal de propósitos educacionais. O livro em questão era “How to Build a Working Digital Computer”³⁷ de E. Alcosser, J. P. Phillips e A. M. Wolk³⁸; ele ensinava como construir partes operacionais de computador usando materiais normalmente disponíveis em casa, como parafusos, laminas metálicas e cliques de papel. Daí o CT-650, construído por Becker, ser também conhecido como o “Computador de Clipe de Papel”.



Figura Parte VI: Do Circuito Integrado à Lua-29: Arkay CT-650.³⁹ 1/1

Mas ele era mais do que isso, o CT (Computer Trainer) tinha uma unidade de entrada e saída para números decimais, uma unidade de aritmética, uma unidade de controle, memória e um tambor magnético usado para armazenar programas. Além disso, usava uma linguagem de programação flexível e fácil de aprender.

O computador também foi conhecido como Arkay CT-650, já que a empresa de Becker se chamava Arkay. O nome vinha da pronuncia inglesa para a sigla “RK”, ou “Radio Kit”, que foi o principal produto de vendas da Arkay.

O CT-650 vendeu poucas unidades, a um preço de cerca de 1000 dólares. Entre outras características, ele tinha uma versão destinada a deficientes visuais. Nessa versão, as saídas da máquina eram representadas em pequenas lâmpadas que se aqueciam

³⁷ Tradução: Como construir um computador digital que funciona.

³⁸ Disponível em: https://archive.org/stream/howtobuildaworkingdigitalcomputer_jun67/HowToBuildAWorkingDigitalComputer_Jun67#page/n7/mode/2up

³⁹ Image courtesy of vintagecomputer.net. Disponível em: <http://www.vintagecomputer.net/pictures/2014/SystemSource/>

ligeiramente, permitindo a uma pessoa cega “ler” a saída a partir do toque dos dedos nessas lâmpadas.

7.36. RS-232C – 1969

A medida que computadores começaram a ser conectados a periféricos tornou-se necessário padronizar os plugues de conexão de forma a evitar que toda a vez fios tivessem que ser soldados ou cortados ou mesmo que transformadores de tensão tivessem que ser usados. O padrão de conexão que perdurou dos anos 1960 até bem recentemente foi originado em 1962 na EIA (Electronic Industries Association – hoje “Electronic Industries Alliance”), uma organização de comércio e padronização. Eles projetaram conectores padronizados para interligar terminais com teletipos. O padrão em questão foi chamado de RS-232 e em 1969 uma nova versão, chamada RS-232C (Figura Parte VI: Do Circuito Integrado à Lua-30) tornou-se padrão para interconexão de computadores e seus periféricos a qual só foi suplantada a partir dos anos 1990 com a invenção das portas USB.

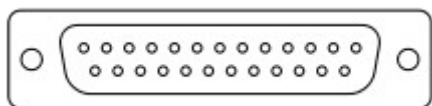


Figura Parte VI: Do Circuito Integrado à Lua-30: Conector DB-25 conforme descrito no padrão RS-232.⁴⁰ 1/3

Porém, o padrão acabou sendo usado em situações que iam muito além da especificação original, que acabou ficando incompleta em alguns casos. Assim, ao longo do tempo, problemas de interpretação do padrão acabaram levando a incompatibilidades que precisavam ser resolvidas.

O padrão definia vários tipos de conectores com números de pinos variados. Ele estabelecia o tamanho e posicionamento de todos pinos, bem como a função de cada um e o tipo de corrente ou sinal que ele deveria usar.

Em comparação com USB, as interfaces RS232 são mais lentas e difíceis de implementar, mas elas permitem conexões com cabos mais longos do que as que são possíveis com cabos USB. Assim, embora hoje computadores pessoais praticamente não usem mais essa tecnologia, ela ainda é importante em aplicações especiais.

7.37. Impressora Laser – 1969

Antes de falar na invenção da impressora laser precisamos voltar no tempo até 1938 quando foi inventado o processo de fotocópia. Chester Carlson (Estados Unidos, 1906-1968) criou nesse ano um processo que ele chamou de “Xerografia”, que em grego significa “escrita a seco”. Ele conseguiu uma patente pela invenção em 1939, mas somente em 1947 convenceu uma empresa a produzir a máquina. Essa empresa chamava-se Haloid, mas mais tarde mudou seu próprio nome em função do processo que a tornou uma gigante no cenário mundial e assim ela passou a chamar-se “Xerox”.

⁴⁰ "DB-25" by Abisys - DSubminiatures.svg + self-made. Licenced under CC BY-SA 3.0 via Wikimedia Commons
<https://commons.wikimedia.org/wiki/File:DB-25.svg#/media/File:DB-25.svg>

A Xerox, portanto, fabricava fotocopiadoras e detinha a patente dessa invenção. Em 1967, um de seus jovens pesquisadores, chamado Gary K. Starkweather (Estados Unidos, 1938) teve uma ideia inusitada: e se ao invés de simplesmente copiar um original, nós ligássemos a fotocopiadora em um computador que poderia *produzir* o original? Com essa ideia nascia a impressora laser.

Apesar de não encontrar encorajamento por parte de seus superiores, ele insistiu no projeto e em 1969 construiu o primeiro protótipo com uma fotocopiadora modificada, com um raio laser que balançava de um lado para o outro de um papel que era preso a um tambor giratório. Um computador era usado para ligar e desligar o raio laser, produzindo dessa forma impressões em partes do papel, as quais seriam reconhecidas como letras e desenhos. Segundo consta, o hardware ficou pronto em uma ou duas semanas, mas o software levou 3 meses para ficar pronto.

Em 1970, quando a Xerox fundou o centro de pesquisas PARC, Starkweather foi trabalhar lá, e o projeto da impressora laser acabou sendo uma das mais importantes contribuições do grupo, sendo que a primeira impressora operacional, a SLOT (Scanned Laser Output Terminal) ficou pronta já em 1971.

Após a produção do software para geração de caracteres, o sistema completo ficou conhecido como EARS (Ethernet, Alto, Research character generator, Scanned laser output terminal), que em seguida tornou-se a primeira impressora laser comercialmente disponível, a Xerox 9700, lançada no mercado em 1977 e mostrada na **Figura Parte VI: Do Circuito Integrado à Lua-31**.



Figura Parte VI: Do Circuito Integrado à Lua-31: Xerox 9700.⁴¹ 1/1

O princípio de funcionamento da impressora laser consiste na produção de imagens em um cilindro eletrostático. A fotocondutividade desse cilindro faz com que as áreas não expostas à luz do laser atraiam uma tinta em pó muito fina, chamada de “toner”. O cilindro transfere essa tinta ao papel através de contato direto e um processo de aquecimento conhecido como “acabamento” faz a fixação da tinta no papel. A Figura Parte VI: Do Circuito Integrado à Lua-32 mostra esquematicamente como isso é feito.

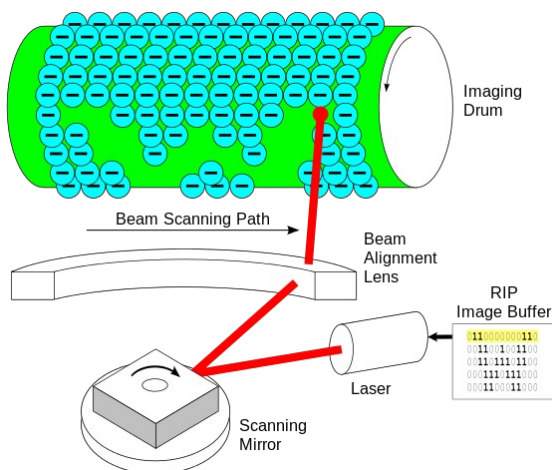


Figura Parte VI: Do Circuito Integrado à Lua-32: Processo de impressão a laser.⁴² 1/2

A Xerox, infelizmente não percebeu que a principal fonte de renda desse negócio não estava nas impressoras, mas no toner e no papel especial. Assim, ela quase foi colocada fora do negócio pela HP, que passou a produzir impressoras muito mais baratas e lucrar com a venda dos insumos. A HP LaserJet, lançada em 1984 a um custo bastante acessível tornou-se praticamente sinônimo de impressora laser.

7.38. Apollo Guidance Computer – 1969

Em 1969 o homem chegou à Lua e junto com ele foi um computador chamado AGC, ou Apollo Guidance Computer (Figura Parte VI: Do Circuito Integrado à Lua-33).

⁴¹ Cortesia de Digibarn Computer Museum e Xerox Corp. Disponível em: <http://www.digibarn.com/collections/printers/xerox-9700/>

⁴² "Laser printer-Writing-es" by Toniperis - Own work. Licenced under CC BY-SA 3.0 via Wikimedia Commons - https://commons.wikimedia.org/wiki/File:Laser_printer-Writing-es.svg#/media/File:Laser_printer-Writing-es.svg

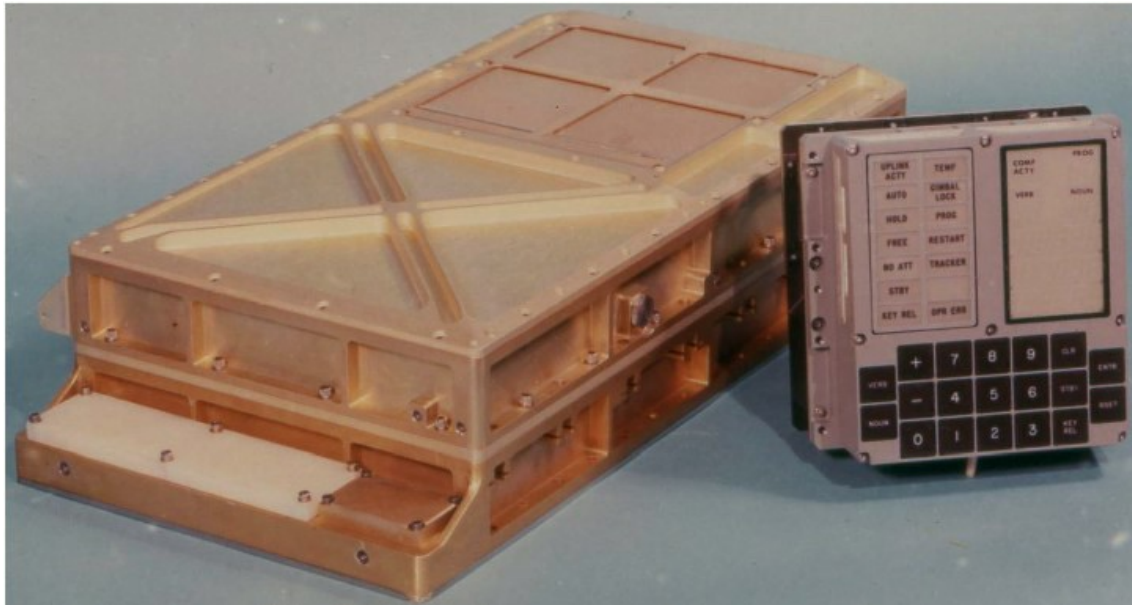


Figura Parte VI: Do Circuito Integrado à Lua-33: Apollo Guidance Computer.⁴³

O AGC tinha palavra de 16 bits sendo um de paridade. A maior parte do software armazenada nele, 36 KB, estava em memória estática (ROM) que só podia ser lida. Apenas uma pequena memória, 2 KB, que podia ser lida e escrita (RAM), estava disponível. O AGC foi um computador baseado em circuitos integrados desenvolvido pelo Laboratório de Instrumentação do MIT.

Nas naves do projeto Apollo, havia 2 AGCs, um no módulo de comando e outro no módulo lunar. Ambos eram responsáveis pela navegação das naves.

O AGC foi projetado para responder a interrupções de hardware, ou seja, de tempos em tempos o processamento podia ser interrompido para executar alguma ação; depois o processamento era retomado. Havia interrupções para atualizar o display do usuário, interrupções para indicar falhas de hardware ou alarmes, para indicar que o usuário pressionou uma tecla, para atualizar o relógio de tempo real e para indicar que uma palavra de 16 bits foi carregada no AGC.

O software para o AGC foi programado em Assembly. Havia um sistema operacional simples chamado Exec que permitia multiprogramação, ou seja, vários programas (até 8) rodarem ao mesmo tempo. O sistema funcionava assim: de tempos em tempos um programa que estivesse rodando tinha que devolver o controle da CPU ao Exec para que ele verificasse se um programa de prioridade mais alta estava esperando para rodar. O programa com a mais baixa prioridade de todos era o Dummy, que não fazia nada a não ser apagar uma luz verde no painel. Quando Dummy estava ativo, isso significava que o computador não tinha outras coisas a fazer e a lâmpada ficava apagada. Mas quando algum processo de maior prioridade tomava conta da CPU, então a lâmpada acendia.

⁴³ "Agc view" by The original uploader was Grabert at German Wikipedia - Transferred from de.wikipedia to Commons by henristosch.. Licensed under Public Domain via Commons - https://commons.wikimedia.org/wiki/File:Agc_view.jpg#/media/File:Agc_view.jpg

O projeto do software para o AGC foi liderado por Margareth Hamilton (Estados Unidos, 1936), diretora da Divisão de Engenharia de Software do Laboratório de Instrumentação do MIT. Ela tinha experiência como programadora no PDP-1 no projeto MAC de Marvin Minsky e trabalhou também no projeto SAGE, para o qual escreveu o software que procurava por aeronaves inimigas.

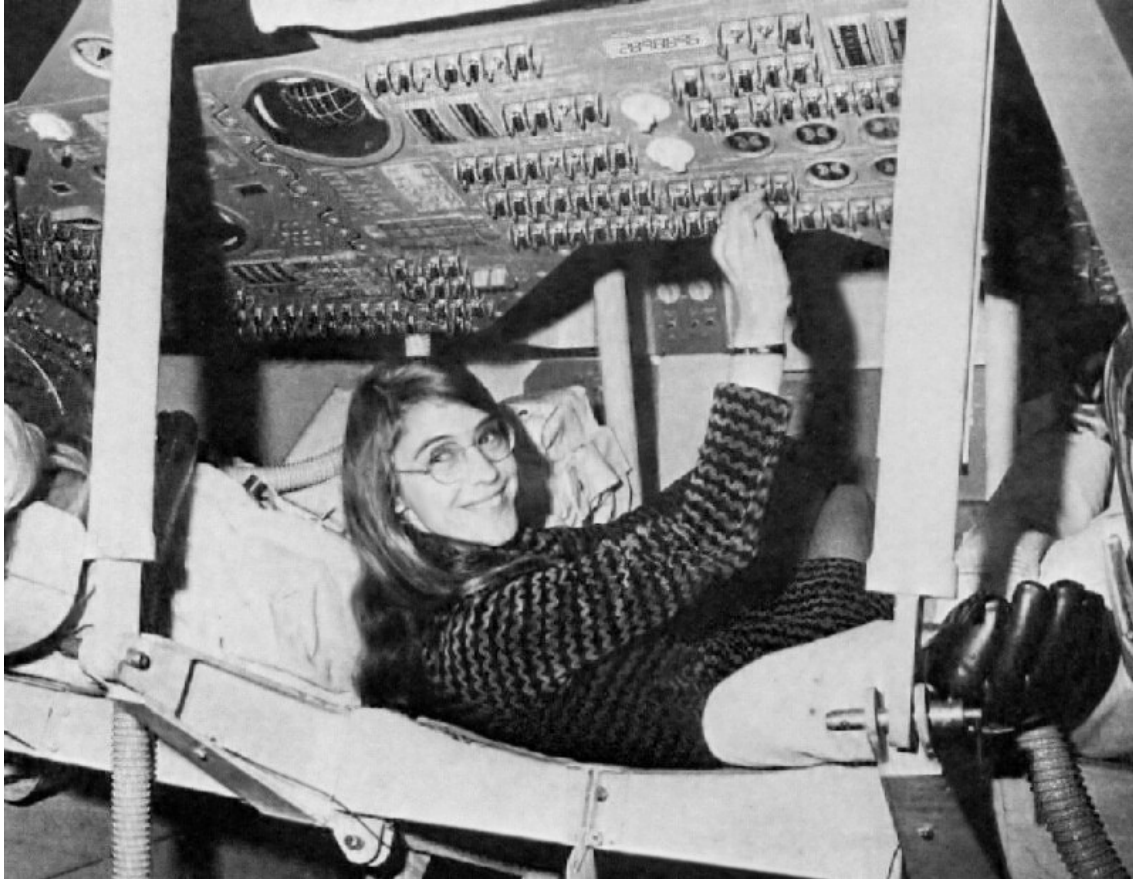


Figura Parte VI: Do Circuito Integrado à Lua-34: Margareth Hamilton no programa Apollo.⁴⁴

O trabalho dela no software de controle do AGC evitou que o primeiro pouso na Lua (Apollo XI) fosse abortado nos instantes finais. Como ela mesma relembra em uma carta para a revista Datamation em 1971: devido a um erro no manual de verificação, um interruptor do radar de encontro foi posicionado no local errado. Isso fez com que ele enviasse sinais errôneos ao computador. O resultado foi que o computador estava sendo solicitado a realizar todas as suas funções normais enquanto recebia uma carga extra de dados espúrios que usava 15% de seu tempo de processador. O software no computador era suficientemente inteligente para reconhecer que ele estava sendo solicitado além de sua capacidade. Ele então enviou um alarme que significava para o astronauta “estou sobrecarregado com mais tarefas do que eu posso realizar agora e vou manter apenas as tarefas mais importantes”, ou seja, as necessárias para a alunissagem. Na verdade, o computador estava programado para fazer mais do que

⁴⁴ "Margaret Hamilton in action" by NASA - http://www.nasa.gov/50th/50th_magazine/scientists.html. Licensed under Public Domain via Commons - https://commons.wikimedia.org/wiki/File:Margaret_Hamilton_in_action.jpg#/media/File:Margaret_Hamilton_in_action.jpg

simplesmente reconhecer situações de erro. Um conjunto completo de programas de recuperação foi incorporado ao software. A ação do software neste caso, foi eliminar os processos de prioridade baixa e reestabelecer os mais importantes. Se o computador não tivesse reconhecido este problema e tomado a ação apropriada, a Apollo XI provavelmente não teria pousado na Lua.

Em 1986, Hamilton recebeu o prêmio Ada Lovelace da Associação da Mulheres na Computação, além de outras distinções. De fato, ela fez muito mais do que simplesmente programar um computador. Ela introduziu conceitos de tolerância a falhas, robustez, garantia de qualidade, detecção de erros, recuperação automática de sistemas, e muitos outros, que fizeram com que fosse considerada uma das mais influentes engenheiras de software da história.

7.39. Até Aqui...

Os anos 1960 foram particularmente férteis para a computação. Como vimos, várias ideias que hoje são fundamentais surgiram nesse período: circuito integrado, compartilhamento de tempo, mouse, computação gráfica, comutação de pacotes, tela sensível ao toque, realidade virtual e aumentada, notebooks, microprocessador, impressora laser e mesmo a rede de computadores, inicialmente com a ARPANET.

Em termos de linguagens de programação, vimos que muitas linguagens importantes surgiram, tais como COBOL, BASIC, MUMPS, Simula 67 e Forth. Knuth inicia nessa década seu tratado épico sobre a arte de programar computadores, em um tempo em que isso nem era reconhecido como profissão. No final dos anos 1960 Margareth Hamilton cria e aperfeiçoa conceitos de engenharia de software que hoje são fundamentais.

Em termos de equipamentos, merece destaque a série PDP, da DEC, que eram computadores interativos, diferentes, portanto, das máquinas de processamento de dados, que até então eram dominantes. Com computadores interativos foi possível iniciar toda uma nova cultura de informática, com videogames, comunicação entre usuários em tempo real e mesmo uma iniciativa na área de inteligência artificial que foi um marco quando ELIZA pela primeira vez simulou um humano conversando com outros humanos.

As ideias do projeto Dynabook da Xerox, rapidamente começaram a evoluir, e nos anos 1970, como veremos, com a evolução do hardware especialmente motivada pela miniaturização dos circuitos integrados, possibilitou o início da era dos computadores pessoais.